Dokumentasi Resmi & Sintaks Kotlin (Kotlin 2.0+ Up-to-Date)

Bahasa: Kotlin | Total Chunk/Dokumen: 150 | Disiapkan untuk RAG System

[kotlin_001] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Overview

Sumber: ht

[//]: # (title: Basic syntax overview)
This is a collection of basic syntax elements with examples. At the end of every section, you'll find a link to a detailed description of the related topic.
You can also learn all the Kotlin essentials with the free Kotlin Core track by JetBrains Academy.

[kotlin_002] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Package definition and imports

Sumber: ht

Package definition and imports
Package specification should be at the top of the source file:
It is not required to match directories and packages: source files can be placed arbitrarily in the file system.
See Packages.

Contoh Kode / Perintah CLI:

```
package my.demo

import kotlin.text.*

// ...
```

[kotlin_003] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Program entry point

Sumber: ht

Program entry point
An entry point of a Kotlin application is the `main` function:
{kotlin-runnable="true" id="kotlin-basic-syntax-hello-world"}
Another form of `main` accepts a variable number of `String` arguments:
{kotlin-runnable="true"}

Contoh Kode / Perintah CLI:

```
fun main() {
    println("Hello world!")
}

// ----------------------------------------

fun main(args: Array<String>) {
    println(args.contentToString())
}
```

[kotlin_004] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Print to the standard output

Sumber: ht

Print to the standard output
`print` prints its argument to the standard output:
{kotlin-runnable="true" id="kotlin-basic-syntax-print"}
`println` prints its arguments and adds a line break, so that the next thing you print appears on the next line:
{kotlin-runnable="true" id="kotlin-basic-syntax-println"}

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    print("Hello ")
    print("world!")
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    println("Hello world!")
    println(42)
//sampleEnd
}
```

[kotlin_005] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Read from the standard input

Sumber: ht

Read from the standard input
The `readln()` function reads from the standard input. This function reads the entire line the user enters as a string.
You can use the `println()`, `readln()`, and `print()` functions together to print messages requesting
and showing user input:
For more information, see Read standard input.

Contoh Kode / Perintah CLI:

```
// Prints a message to request input
println("Enter any word: ")

// Reads and stores the user input. For example: Happiness
val yourWord = readln()

// Prints a message with the input
print("You entered the word: ")
print(yourWord)
// You entered the word: Happiness
```

[kotlin_006] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Functions

Sumber: ht

Functions
A function with two `Int` parameters and `Int` return type:
{kotlin-runnable="true" id="kotlin-basic-syntax-return-int"}
A function body can be an expression. Its return type is inferred:
{kotlin-runnable="true" id="kotlin-basic-syntax-function-expression"}
A function that returns no meaningful value:
{kotlin-runnable="true" id="kotlin-basic-syntax-return-unit"}
`Unit` return type can be omitted:
{kotlin-runnable="true" id="kotlin-basic-syntax-function-omit-unit"}
See Functions.

Contoh Kode / Perintah CLI:

```
//sampleStart
fun sum(a: Int, b: Int): Int {
    return a + b
}
//sampleEnd

fun main() {
    print("sum of 3 and 5 is ")
    println(sum(3, 5))
}

// ----------------------------------------

//sampleStart
fun sum(a: Int, b: Int) = a + b
//sampleEnd

fun main() {
    println("sum of 19 and 23 is ${sum(19, 23)}")
}

// ----------------------------------------

//sampleStart
fun printSum(a: Int, b: Int): Unit {
    println("sum of $a and $b is ${a + b}")
}
//sampleEnd

fun main() {
    printSum(-1, 8)
}
```

[kotlin_007] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Variables

Sumber: ht

Variables
In Kotlin, you declare a variable starting with a keyword, `val` or `var`, followed by the name of the variable.
Use the `val` keyword to declare variables that are assigned a value only once. These are immutable, read-only local variables that can't be reassigned a different value
after initialization:

{kotlin-runnable="true" id="kotlin-basic-syntax-val"}

Use the `var` keyword to declare variables that can be reassigned. These are mutable variables, and you can change their values after initialization:

{kotlin-runnable="true" id="kotlin-basic-syntax-var"}

Kotlin supports type inference and automatically identifies the data type of a declared variable. When declaring a variable, you can omit the type after the variable name:

{kotlin-runnable="true" id="kotlin-basic-syntax-inference"}

You can use variables only after initializing them. You can either initialize a variable at the moment of declaration or declare a variable first and initialize it later.

In the second case, you must specify the data type:

{kotlin-runnable="true" id="kotlin-basic-syntax-initialize"}

You can declare variables at the top level:

{kotlin-runnable="true" id="kotlin-basic-syntax-variable-top-level"}

For information about declaring properties, see Properties.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Declares the variable x and initializes it with the value of 5
    val x: Int = 5
    // 5
//sampleEnd
    println(x)
}

// ----------------------------------------

fun main() {
//sampleStart
    // Declares the variable x and initializes it with the value of 5
    var x: Int = 5
    // Reassigns a new value of 6 to the variable x
    x += 1
    // 6
//sampleEnd
    println(x)
}

// ----------------------------------------

fun main() {
//sampleStart
    // Declares the variable x with the value of 5;`Int` type is inferred
    val x = 5
    // 5
//sampleEnd
    println(x)
}
```

[kotlin_008] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Creating classes and instances

Sumber: ht

Creating classes and instances

To define a class, use the `class` keyword:

Properties of a class can be listed in its declaration or body:

The default constructor with parameters listed in the class declaration is available automatically:

{kotlin-runnable="true" id="kotlin-basic-syntax-class-constructor"}
Inheritance between classes is declared by a colon (`:`). Classes are `final` by default; to make a class inheritable, mark it as `open`:
For more information about constructors and inheritance, see Classes and Objects and instances.

Contoh Kode / Perintah CLI:

```
class Shape

// ----------------------------------------

class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}

// ----------------------------------------

class Rectangle(val height: Double, val length: Double) {
    val perimeter = (height + length) * 2
}
fun main() {
    val rectangle = Rectangle(5.0, 2.0)
    println("The perimeter is ${rectangle.perimeter}")
}
```

[kotlin_009] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Comments

Sumber: ht

Comments
Just like most modern languages, Kotlin supports single-line (or _end-of-line_) and multi-line (_block_) comments:
Block comments in Kotlin can be nested:
See Documenting Kotlin Code for information on the documentation comment syntax.

Contoh Kode / Perintah CLI:

```
// This is an end-of-line comment

/* This is a block comment
   on multiple lines. */

// ----------------------------------------

/* The comment starts here
/* contains a nested comment */
and ends here. */
```

[kotlin_010] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): String templates

Sumber: ht

String templates
{kotlin-runnable="true" id="kotlin-basic-syntax-string-templates"}
See String templates for details.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    var a = 1
    // simple name in template:
    val s1 = "a is $a"
```

```
    a = 2
    // arbitrary expression in template:
    val s2 = "${s1.replace("is", "was")}, but now is $a"
//sampleEnd
    println(s2)
}
```

[kotlin_011] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Conditional expressions

Sumber: ht

Conditional expressions
{kotlin-runnable="true" id="kotlin-basic-syntax-conditional-expressions"}
In Kotlin, `if` can also be used as an expression:
{kotlin-runnable="true" id="kotlin-basic-syntax-if-expression"}
See `if`-expressions.

Contoh Kode / Perintah CLI:

```
//sampleStart
fun maxOf(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
//sampleEnd

fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}

// ----------------------------------------

//sampleStart
fun maxOf(a: Int, b: Int) = if (a > b) a else b
//sampleEnd

fun main() {
    println("max of 0 and 42 is ${maxOf(0, 42)}")
}
```

[kotlin_012] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): for loop

Sumber: ht

for loop
{kotlin-runnable="true" id="kotlin-basic-syntax-for-loop"}
or:
{kotlin-runnable="true" id="kotlin-basic-syntax-for-loop-indices"}
See for loop.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    val items = listOf("apple", "banana", "kiwifruit")
    for (item in items) {
        println(item)
```

```
    }
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    val items = listOf("apple", "banana", "kiwifruit")
    for (index in items.indices) {
        println("item at $index is ${items[index]}")
    }
//sampleEnd
}
```

[kotlin_013] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): while loop

Sumber: ht

while loop
{kotlin-runnable="true" id="kotlin-basic-syntax-while-loop"}
See while loop.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    val items = listOf("apple", "banana", "kiwifruit")
    var index = 0
    while (index < items.size) {
        println("item at $index is ${items[index]}")
        index++
    }
//sampleEnd
}
```

[kotlin_014] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): when expression

Sumber: ht

when expression
{kotlin-runnable="true" id="kotlin-basic-syntax-when-expression"}
See when expressions and statements.

Contoh Kode / Perintah CLI:

```
//sampleStart
fun describe(obj: Any): String =
    when (obj) {
        1          -> "One"
        "Hello"    -> "Greeting"
        is Long    -> "Long"
        !is String -> "Not a string"
        else       -> "Unknown"
    }
//sampleEnd

fun main() {
    println(describe(1))
    println(describe("Hello"))
    println(describe(1000L))
    println(describe(2))
```

```
    println(describe("other"))
}
```

[kotlin_015] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Ranges

Sumber: ht

Ranges
Check if a number is within a range using `in` operator:
{kotlin-runnable="true" id="kotlin-basic-syntax-range-in"}
Check if a number is out of range:
{kotlin-runnable="true" id="kotlin-basic-syntax-out-of-range"}
Iterate over a range:
{kotlin-runnable="true" id="kotlin-basic-syntax-iterate-range"}
Or over a progression:
{kotlin-runnable="true" id="kotlin-basic-syntax-iterate-progression"}
See Ranges and progressions.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    val x = 10
    val y = 9
    if (x in 1..y+1) {
        println("fits in range")
    }
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    val list = listOf("a", "b", "c")

    if (-1 !in 0..list.lastIndex) {
        println("-1 is out of range")
    }
    if (list.size !in list.indices) {
        println("list size is out of valid list indices range, too")
    }
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    for (x in 1..5) {
        print(x)
    }
//sampleEnd
}
```

[kotlin_016] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Collections

Sumber: ht

Collections
Iterate over a collection:
{kotlin-runnable="true" id="kotlin-basic-syntax-iterate-collection"}
Check if a collection contains an object using `in` operator:
{kotlin-runnable="true" id="kotlin-basic-syntax-collection-in"}
Use lambda expressions to filter and map collections:
{kotlin-runnable="true" id="kotlin-basic-syntax-collection-filter-map"}
See Collections overview.

Contoh Kode / Perintah CLI:

```
fun main() {
    val items = listOf("apple", "banana", "kiwifruit")
//sampleStart
    for (item in items) {
        println(item)
    }
//sampleEnd
}

// ----------------------------------------

fun main() {
    val items = setOf("apple", "banana", "kiwifruit")
//sampleStart
    when {
        "orange" in items -> println("juicy")
        "apple" in items -> println("apple is fine too")
    }
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    val fruits = listOf("banana", "avocado", "apple", "kiwifruit")
    fruits
      .filter { it.startsWith("a") }
      .sortedBy { it }
      .map { it.uppercase() }
      .forEach { println(it) }
//sampleEnd
}
```

[kotlin_017] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Nullable values and null checks

Sumber: ht

Nullable values and null checks
A reference must be explicitly marked as nullable when a `null` value is possible. Nullable type names have `?` at the end.
For example, `Int?`.
Return `null` if `str` does not hold an integer:

Use a function returning nullable value:
{kotlin-runnable="true" id="kotlin-basic-syntax-function-nullable-value"}
or:
{kotlin-runnable="true" id="kotlin-basic-syntax-function-null-check"}
See Null-safety.

Contoh Kode / Perintah CLI:

```
fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

// -----------------------------------------

fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

//sampleStart
fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)

    // Using `x * y` yields error because they may hold nulls.
    if (x != null && y != null) {
        // x and y are automatically cast to non-nullable after null check
        println(x * y)
    }
    else {
        println("'$arg1' or '$arg2' is not a number")
    }
}
//sampleEnd

fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("a", "b")
}

// -----------------------------------------

fun parseInt(str: String): Int? {
    return str.toIntOrNull()
}

fun printProduct(arg1: String, arg2: String) {
    val x = parseInt(arg1)
    val y = parseInt(arg2)

//sampleStart
    // ...
    if (x == null) {
        println("Wrong number format in arg1: '$arg1'")
        return
    }
    if (y == null) {
        println("Wrong number format in arg2: '$arg2'")
        return
    }

    // x and y are automatically cast to non-nullable after null check
```

```
    println(x * y)
//sampleEnd
}

fun main() {
    printProduct("6", "7")
    printProduct("a", "7")
    printProduct("99", "b")
}
```

[kotlin_018] Kotlin - Basic Syntax & Idioms

Basic Syntax (Variables, Functions, String templates, Collections): Type checks and automatic casts

Sumber: ht

Type checks and automatic casts

The `is` operator checks if an expression is an instance of a type.

If an immutable local variable or property is checked for a specific type, there's no need to cast it explicitly:

{kotlin-runnable="true" id="kotlin-basic-syntax-is-operator"}

or:

{kotlin-runnable="true" id="kotlin-basic-syntax-is-operator-expression"}

or even:

{kotlin-runnable="true" id="kotlin-basic-syntax-is-operator-logic"}

See Classes and Type casts.

Contoh Kode / Perintah CLI:

```
//sampleStart
fun getStringLength(obj: Any): Int? {
    if (obj is String) {
        // `obj` is automatically cast to `String` in this branch
        return obj.length
    }

    // `obj` is still of type `Any` outside of the type-checked branch
    return null
}
//sampleEnd

fun main() {
    fun printLength(obj: Any) {
         println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The object is
not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf(Any()))
}

// ----------------------------------------

//sampleStart
fun getStringLength(obj: Any): Int? {
    if (obj !is String) return null

    // `obj` is automatically cast to `String` in this branch
    return obj.length
}
//sampleEnd
```

```
fun main() {
    fun printLength(obj: Any) {
         println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The object is
not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength(1000)
    printLength(listOf(Any()))
}

// ------------------------------------------

//sampleStart
fun getStringLength(obj: Any): Int? {
    // `obj` is automatically cast to `String` on the right-hand side of `&&`
    if (obj is String && obj.length >= 0) {
        return obj.length
    }

    return null
}
//sampleEnd

fun main() {
    fun printLength(obj: Any) {
         println("Getting the length of '$obj'. Result: ${getStringLength(obj) ?: "Error: The object is
not a string"} ")
    }
    printLength("Incomprehensibilities")
    printLength("")
    printLength(1000)
}
```

[kotlin_019] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Overview

Sumber: ht

[//]: # (title: Conditions and loops)
Kotlin gives you flexible tools to control your program's flow. Use `if`, `when`, and loops to define clear, expressive logic for your conditions.

[kotlin_020] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): If expression

Sumber: ht

If expression
To use `if` in Kotlin, add the condition to check within parentheses `()` and the action to take if the result is true within curly braces `{}`. You can use `else` and `else if` for additional branches and checks.
You can also write `if` as an expression, which lets you assign its returned value directly to a variable.
In this form, an `else` branch is required. The `if` expression serves the same purpose as the ternary operator (`condition ? then : else`) found in other languages.
For example:
{kotlin-runnable="true" id="if-else-if-kotlin"}
Each branch in an `if` expression can be a block, where the value of the last expression becomes the result:
{kotlin-runnable="true" id="if-else-blocks-kotlin"}

Contoh Kode / Perintah CLI:

```
fun main() {
    val heightAlice = 160
```

```
    val heightBob = 175

    //sampleStart
    var taller = heightAlice
    if (heightAlice < heightBob) taller = heightBob

    // Uses an else branch
    if (heightAlice > heightBob) {
        taller = heightAlice
    } else {
        taller = heightBob
    }

    // Uses if as an expression
    taller = if (heightAlice > heightBob) heightAlice else heightBob

    // Uses else if as an expression:
    val heightLimit = 150
     val heightOrLimit = if (heightLimit > heightAlice) heightLimit else if (heightAlice > heightBob)
heightAlice else heightBob

    println("Taller height is $taller")
    // Taller height is 175
    println("Height or limit is $heightOrLimit")
    // Height or limit is 175
    //sampleEnd
}

// ----------------------------------------

fun main() {
    //sampleStart
    val heightAlice = 160
    val heightBob = 175

    val taller = if (heightAlice > heightBob) {
        print("Choose Alice\n")
        heightAlice
    } else {
        print("Choose Bob\n")
        heightBob
    }

    println("Taller height is $taller")
    //sampleEnd
}
```

[kotlin_021] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): When expressions and statements

Sumber: ht

When expressions and statements
`when` is a conditional expression that runs code based on multiple possible values or conditions. It's
similar to the `switch` statement in Java, C, and other languages. `when` evaluates its argument and compares the result
against each branch in order until one branch condition is satisfied. For example:
{kotlin-runnable="true" id="kotlin-conditions-when-statement"}
You can use `when` either as an **expression** or a **statement**. As an expression, `when` returns a value you can

use
later in your code. As a statement, `when` completes an action without returning a result:
Expression
Statement
Secondly, you can use `when` with or without a subject. The behavior stays the same either way. Using a subject usually
makes your code more readable and maintainable because it clearly shows what you're checking.
With subject x
Without subject
How you use `when` determines whether you need to cover all possible cases in your branches. Covering all possible cases
is called being _exhaustive_.

Contoh Kode / Perintah CLI:

```
fun main() {
    //sampleStart
    val userRole = "Editor"
    when (userRole) {
        "Viewer" -> print("User has read-only access")
        "Editor" -> print("User can edit content")
        else -> print("User role is not recognized")
    }
    // User can edit content
    //sampleEnd
}

// ----------------------------------------

// Returns a string assigned to the
// text variable
val text = when (x) {
    1 -> "x == 1"
    2 -> "x == 2"
    else -> "x is neither 1 nor 2"
}

// ----------------------------------------

// Returns no result but triggers a
// print statement
when (x) {
    1 -> print("x == 1")
    2 -> print("x == 2")
    else -> print("x is neither 1 nor 2")
}
```

[kotlin_022] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Statements

Sumber: ht

Statements
If you use `when` as a statement, you don't need to cover all possible cases. In this example, some cases aren't covered,
so no branch is triggered. However, no error occurs:
{kotlin-runnable="true" id="kotlin-when-statement"}
Just like with `if`, each branch can be a block, and its value is the value of the last expression in the block.

Contoh Kode / Perintah CLI:

```
fun main() {
```

```
    //sampleStart
    val deliveryStatus = "OutForDelivery"
    when (deliveryStatus) {
        // Not all cases are covered
        "Pending" -> print("Your order is being prepared")
        "Shipped" -> print("Your order is on the way")
    }
    //sampleEnd
}
```

[kotlin_023] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Expressions

Sumber: ht

Expressions
If you use `when` as an expression, you **must** cover all possible cases. The value of the first matching branch becomes
the value of the overall expression. If you don't cover all cases, the compiler throws an error.
If your `when` expression has a subject, you can use an `else` branch to make sure that all possible cases are covered, but
it isn't mandatory. For example, if your subject is a `Boolean`, `enum` class, `sealed` class,
or one of their nullable counterparts, you can cover all cases without an `else` branch:
{kotlin-runnable="true" id="kotlin-when-expression-subject"}
> To simplify `when` expressions and reduce repetition, try out context-sensitive resolution (currently in preview).
> This feature allows you to omit the type name when using enum entries or sealed class members in `when` expressions if the expected type is known.
>
> For more information, see Preview of context-sensitive resolution or the related KEEP proposal.
>
{style="tip"}
If your `when` expression **doesn't** have a subject, you **must** have an `else` branch or the compiler throws an error.
The `else` branch is evaluated when none of the other branch conditions are satisfied:
{kotlin-runnable="true" id="kotlin-when-no-subject"}

Contoh Kode / Perintah CLI:

```
import kotlin.random.Random
//sampleStart
enum class Bit {
    ZERO, ONE
}

fun getRandomBit(): Bit {
    return if (Random.nextBoolean()) Bit.ONE else Bit.ZERO
}

fun main() {
    val numericValue = when (getRandomBit()) {
        // No else branch is needed because all cases are covered
        Bit.ZERO -> 0
        Bit.ONE -> 1
    }

    println("Random bit as number: $numericValue")
    // Random bit as number: 0
    //sampleEnd
}
```

```
// ----------------------------------------

fun main() {
    //sampleStart
    val localFileSize = 1200
    val remoteFileSize = 1200

    val message = when {
        localFileSize > remoteFileSize -> "Local file is larger than remote file"
        localFileSize < remoteFileSize -> "Local file is smaller than remote file"
        else -> "Local and remote files are the same size"
    }

    println(message)
    // Local and remote files are the same size
    //sampleEnd
}
```

[kotlin_024] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Other ways to use when

Sumber: ht

Other ways to use when
`when` expressions and statements offer different ways to simplify your code, handle multiple conditions, and perform type checks.
Group multiple conditions into a single branch using commas:
{kotlin-runnable="true" id="kotlin-when-multiple-cases"}
Use expressions that evaluate to `true` or `false` as branch conditions:
{kotlin-runnable="true" id="kotlin-when-branch-expression"}
Check whether a value is or isn't contained in a range or collection using the `in` or `!in` keywords:
{kotlin-runnable="true" id="kotlin-when-ranges"}
Check a value's type using the `is` or `!is` keywords. Due to smart casts, you can access the member functions and properties of the type directly:
{kotlin-runnable="true" id="kotlin-when-type-checks"}
Use `when` instead of a traditional `if`-`else` `if` chain.
Without a subject, the branch conditions are simply boolean expressions. The first branch with a `true` condition runs:
{kotlin-runnable="true" id="kotlin-when-replace-if"}
Finally, capture the subject in a variable by using the following syntax:
{kotlin-runnable="true" id="kotlin-when-capture-subject"}
The scope of a variable introduced as the subject is restricted to the body of the `when` expression or statement.

Contoh Kode / Perintah CLI:

```
fun main() {
    val ticketPriority = "High"
    //sampleStart
    when (ticketPriority) {
        "Low", "Medium" -> print("Standard response time")
        else -> print("High-priority handling")
    }
    //sampleEnd
}

// ----------------------------------------

fun main() {
    val storedPin = "1234"
    val enteredPin = 1234
```

```
    //sampleStart
    when (enteredPin) {
        // Expression
        storedPin.toInt() -> print("PIN is correct")
        else -> print("Incorrect PIN")
    }
    //sampleEnd
}

// ----------------------------------------

fun main() {
    val x = 7
    val validNumbers = setOf(15, 16, 17)

    //sampleStart
    when (x) {
        in 1..10 -> print("x is in the range")
        in validNumbers -> print("x is valid")
        !in 10..20 -> print("x is outside the range")
        else -> print("none of the above")
    }
    //sampleEnd
}
```

[kotlin_025] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Guard conditions {id="guard-conditions-in-when-expressions"}

Sumber: ht

Guard conditions {id="guard-conditions-in-when-expressions"}
Guard conditions allow you to include more than one condition to the branches of a `when` expression or statement, making complex
control flow more explicit and concise. You can use guard conditions with `when` as long as it has a subject.
Place a guard condition after the primary condition in the same branch, separated by `if`:
{kotlin-runnable="true" kotlin-min-compiler-version="2.2" id="kotlin-when-guard-conditions"}
You can't use guard conditions when you have multiple conditions separated by a comma. For example:
In a single `when` expression or statement, you can combine branches with and without guard conditions.
The code in a branch with a guard condition runs only if both the primary condition and the guard condition evaluate to `true`.
If the primary condition doesn't match, the guard condition isn't evaluated.
Since `when` statements don't need to cover all cases, using guard conditions in `when` statements without an `else` branch means that if no conditions match, no code is run.
Unlike statements, `when` expressions must cover all cases. If you use guard conditions in `when` expressions without an `else` branch,
the compiler requires you to handle every possible case to avoid runtime errors.
Combine multiple guard conditions within a single branch using the boolean operators `&&` (AND) or `||` (OR).
Use parentheses around the boolean expressions to avoid confusion:
Guard conditions also support `else if`:

Contoh Kode / Perintah CLI:

```
sealed interface Animal {
    data class Cat(val mouseHunter: Boolean) : Animal
    data class Dog(val breed: String) : Animal
}
```

```
fun feedDog() = println("Feeding a dog")
fun feedCat() = println("Feeding a cat")

//sampleStart
fun feedAnimal(animal: Animal) {
    when (animal) {
        // Branch with only primary condition
        // Calls feedDog() when animal is Dog
        is Animal.Dog -> feedDog()
        // Branch with both primary and guard conditions
        // Calls feedCat() when animal is Cat and not mouseHunter
        is Animal.Cat if !animal.mouseHunter -> feedCat()
        // Prints "Unknown animal" if none of the above conditions match
        else -> println("Unknown animal")
    }
}

fun main() {
    val animals = listOf(
        Animal.Dog("Beagle"),
        Animal.Cat(mouseHunter = false),
        Animal.Cat(mouseHunter = true)
    )

    animals.forEach { feedAnimal(it) }
    // Feeding a dog
    // Feeding a cat
    // Unknown animal
}
//sampleEnd

// ----------------------------------------

0, 1 -> print("x == 0 or x == 1")

// ----------------------------------------

when (animal) {
    is Animal.Cat if (!animal.mouseHunter && animal.hungry) -> feedCat()
}
```

[kotlin_026] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Bytecode generation on the JVM

Sumber: ht

Bytecode generation on the JVM

When you compile Kotlin code for JVM 21 or later, the compiler generates an `invokedynamic`
instruction for eligible `when` expressions. This produces smaller bytecode, similar to the bytecode produced by Java `switch` statements.

The compiler uses `invokedynamic` with the `SwitchBootstraps.typeSwitch()`
method when all the following conditions are met:

* All conditions except for `else` are `is` or `null` checks.
* The `when` expression doesn't contain guard conditions (`if`).
* The conditions don't include types that can't be type-checked directly, such as mutable Kotlin collections (`MutableList`) or function types (`kotlin.Function1`, `kotlin.Function2`, and so on).
* The `when` expression has at least two conditions besides `else`.
* All branches check the same subject of the `when` expression.

For example:
The `when (shape)` expression here compiles to a single `invokedynamic` type switch instead of multiple `instanceof` checks
in the bytecode.

Contoh Kode / Perintah CLI:

```
open class Shape

class Circle : Shape()
class Rectangle : Shape()
class Triangle : Shape()

fun countCorners(shape: Shape) = when (shape) {
    is Circle -> 0
    is Rectangle -> 4
    is Triangle -> 3
    else -> -1
}
```

[kotlin_027] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): For loops

Sumber: ht

For loops
Use the `for` loop to iterate through a collection, array, or range:
The body of a `for` loop can be a block with curly braces `{}`.
{kotlin-runnable="true" id="kotlin-for-loop"}

Contoh Kode / Perintah CLI:

```
for (item in collection) print(item)

// ----------------------------------------

fun main() {
    val shoppingList = listOf("Milk", "Bananas", "Bread")
    //sampleStart
    println("Things to buy:")
    for (item in shoppingList) {
        println("- $item")
    }
    // Things to buy:
    // - Milk
    // - Bananas
    // - Bread
    //sampleEnd
}
```

[kotlin_028] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Ranges

Sumber: ht

Ranges
To iterate over a range of numbers, use a range expression with `..` and `..<` operators:
{kotlin-runnable="true" id="kotlin-for-loop-range"}

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    println("Closed-ended range:")
    for (i in 1..6) {
```

```
        print(i)
    }
    // Closed-ended range:
    // 123456

    println("\nOpen-ended range:")
    for (i in 1..<6) {
        print(i)
    }
    // Open-ended range:
    // 12345

    println("\nReverse order in steps of 2:")
    for (i in 6 downTo 0 step 2) {
        print(i)
    }
    // Reverse order in steps of 2:
    // 6420
//sampleEnd
}
```

[kotlin_029] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Arrays

Sumber: ht

Arrays

If you want to iterate through an array or a list with an index, you can use the `indices` property:

{kotlin-runnable="true" id="kotlin-for-loop-array"}

Alternatively, you can use the `.withIndex()` function from the standard library:

{kotlin-runnable="true" id="kotlin-for-loop-array-index"}

Contoh Kode / Perintah CLI:

```
fun main() {
    val routineSteps = arrayOf("Wake up", "Brush teeth", "Make coffee")
    //sampleStart
    for (i in routineSteps.indices) {
        println(routineSteps[i])
    }
    // Wake up
    // Brush teeth
    // Make coffee
    //sampleEnd
}

// ----------------------------------------

fun main() {
    val routineSteps = arrayOf("Wake up", "Brush teeth", "Make coffee")
    //sampleStart
    for ((index, value) in routineSteps.withIndex()) {
        println("The step at $index is \"$value\"")
    }
    // The step at 0 is "Wake up"
    // The step at 1 is "Brush teeth"
    // The step at 2 is "Make coffee"
    //sampleEnd
}
```

[kotlin_030] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Iterators

Sumber: ht

Iterators

The `for` loop iterates through anything that provides an iterator. Collections provide iterators by default, whereas ranges and arrays are compiled into index-based loops.

You can create your own iterators by providing a member or extension function called `iterator()` that returns an `Iterator<>`.

The `iterator()` function must have a `next()` function and a `hasNext()` function that returns a `Boolean`.

The easiest way to create your own iterator for a class is to inherit from the `Iterable` interface and override the `iterator()`, `next()`, and `hasNext()` functions that are already there. For example:

{kotlin-runnable="true" id="kotlin-for-loop-inherit-iterator"}

> Learn more about interfaces and inheritance.
>

{style="tip"}

Alternatively, you can create the functions from scratch. In this case, add the `operator` keyword to the functions:

{kotlin-runnable="true" id="kotlin-for-loop-iterator-from-scratch"}

Contoh Kode / Perintah CLI:

```
class Booklet(val totalPages: Int) : Iterable<Int> {
    override fun iterator(): Iterator<Int> {
        return object : Iterator<Int> {
            var current = 1
            override fun hasNext() = current <= totalPages
            override fun next() = current++
        }
    }
}

fun main() {
    val booklet = Booklet(3)
    for (page in booklet) {
        println("Reading page $page")
    }
    // Reading page 1
    // Reading page 2
    // Reading page 3
}

// ----------------------------------------

//sampleStart
class Booklet(val totalPages: Int) {
    operator fun iterator(): Iterator<Int> {
        return object {
            var current = 1

            operator fun hasNext() = current <= totalPages
            operator fun next() = current++
        }.let {
            object : Iterator<Int> {
                override fun hasNext() = it.hasNext()
                override fun next() = it.next()
            }
        }
    }
}
//sampleEnd
```

```
fun main() {
    val booklet = Booklet(3)
    for (page in booklet) {
        println("Reading page $page")
    }
    // Reading page 1
    // Reading page 2
    // Reading page 3
}
```

[kotlin_031] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): While loops

Sumber: ht

While loops
`while` and `do-while` loops run the code in their body continuously while the condition is satisfied.
The difference between them is the condition checking time:
* `while` checks the condition and, if it's satisfied, runs the code in its body and then returns to the condition check.
* `do-while` runs the code in its body and then checks the condition. If it's satisfied, the loop repeats. So, the body of `do-while`
runs at least once, regardless of the condition.
For a `while` loop, place the condition to check in parentheses `()` and the body within curly braces `{}`:
{kotlin-runnable="true" id="kotlin-while-loop"}
For a `do-while` loop, place the body within curly braces `{}` first before the condition to check in parentheses `()`:
{kotlin-runnable="true" id="kotlin-do-while-loop"}

Contoh Kode / Perintah CLI:

```
fun main() {
    var carsInGarage = 0
    val maxCapacity = 3
//sampleStart
    while (carsInGarage < maxCapacity) {
        println("Car entered. Cars now in garage: ${++carsInGarage}")
    }
    // Car entered. Cars now in garage: 1
    // Car entered. Cars now in garage: 2
    // Car entered. Cars now in garage: 3

    println("Garage is full!")
    // Garage is full!
//sampleEnd
}

// ----------------------------------------

import kotlin.random.Random

fun main() {
    var roll: Int
//sampleStart
    do {
        roll = Random.nextInt(1, 7)
        println("Rolled a $roll")
    } while (roll != 6)
    // Rolled a 2
    // Rolled a 6

    println("Got a 6! Game over.")
    // Got a 6! Game over.
```

```
//sampleEnd
}
```

[kotlin_032] Kotlin - Control Flow & Pattern Matching

Control Flow (when Expression, if expression, for & while loops, Ranges): Break and continue in loops

Sumber: ht

Break and continue in loops
Kotlin supports traditional `break` and `continue` operators in loops. See Returns and jumps.

[kotlin_033] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Overview

Sumber: ht

[//]: # (title: Null safety)
Null safety is a Kotlin feature designed to significantly reduce the risk of null references, also known as The Billion-Dollar Mistake.
One of the most common pitfalls in many programming languages, including Java, is that accessing a member of a null reference results in a null reference exception. In Java, this would be the equivalent of a `NullPointerException`, or an _NPE_ for short.
Kotlin explicitly supports nullability as part of its type system, meaning you can explicitly declare which variables or properties are allowed to be `null`. Also, when you declare non-null variables, the compiler enforces that these variables cannot hold a `null` value, preventing an NPE.
Kotlin's null safety ensures safer code by catching potential null-related issues at compile time rather than runtime.
This feature improves code robustness, readability, and maintainability by explicitly expressing `null` values, making the code easier to understand and manage.
The only possible causes of an NPE in Kotlin are:
* An explicit call to `throw NullPointerException()`.
* Usage of the not-null assertion operator `!!`.
* Data inconsistency during initialization, such as when:
* An uninitialized `this` available in a constructor is used somewhere else (a "leaking `this`").
* A superclass constructor calling an open member whose implementation in the derived class uses an uninitialized state.
* Java interoperation:
* Attempts to access a member of a `null` reference of a platform type.
* Nullability issues with generic types. For example, a piece of Java code adding `null` into a Kotlin `MutableList`, which would require `MutableList` to handle it properly.
* Other issues caused by external Java code.
> Besides NPE, another exception related to null safety is `Unin

[kotlin_034] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Nullable types and non-nullable types

Sumber: ht

Nullable types and non-nullable types
In Kotlin, the type system distinguishes between types that can hold `null` (nullable types) and those that cannot (non-nullable types). For example, a regular variable of type `String` cannot hold `null`:
{kotlin-runnable="true" validate="false"}
You can safely call a method or access a property on `a`. It's guaranteed not to cause an NPE because `a` is a non-nullable variable.

The compiler ensures that `a` always holds a valid `String` value, so there's no risk of accessing its properties or methods when it's `null`:
{kotlin-runnable="true" validate="false"}
To allow `null` values, declare a variable with a `?` sign right after the variable type. For example, you can declare a nullable string by writing `String?`. This expression makes `String` a type that can accept `null`:
{kotlin-runnable="true"}
If you try accessing `length` directly on `b`, the compiler reports an error. This is because `b` is declared as a nullable variable and can hold `null` values. Attempting to access properties on nullables directly leads to an NPE:
{kotlin-runnable="true" kotlin-min-compiler-version="2.0" validate="false"}
In the example above, the compiler requires you to use safe calls to check for nullability before accessing properties or performing operations. There are several ways to handle nullables:
* Check for `null` with the `if` conditional
* Safe call operator `?.`
* Elvis operator `?:`
* Not-null assertion operator `!!`
* Nullable receiver
* `let` function
* Safe casts `as?`
* Collections of a nullable type
Read the next sections for details and examples of `null` handling tools and techniques.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Assigns a non-null string to a variable
    var a: String = "abc"
    // Attempts to re-assign null to the non-nullable variable
    a = null
    print(a)
    // Null can not be a value of a non-null type String
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    // Assigns a non-null string to a variable
    val a: String = "abc"
    // Returns the length of a non-nullable variable
    val l = a.length
    print(l)
    // 3
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    // Assigns a nullable string to a variable
    var b: String? = "abc"
    // Successfully re-assigns null to the nullable variable
    b = null
    print(b)
    // null
//sampleEnd
```

```
}
```

[kotlin_035] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Check for null with the if conditional

Sumber: ht

Check for null with the if conditional
When working with nullable types, you need to handle nullability safely to avoid an NPE. One way to
handle this is checking for nullability explicitly with the `if` conditional expression.
For example, check whether `b` is `null` and then access `b.length`:
{kotlin-runnable="true"}
In the example above, the compiler performs a smart cast to change the type from nullable `String?` to non-nullable `String`. It also tracks the information about
the check you performed and allows the call to `length` inside the `if` conditional.
More complex conditions are supported as well:
{kotlin-runnable="true"}
Note that the example above only works when the compiler can guarantee that `b` doesn't change between the check and its usage, same as
the smart cast prerequisites.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability first and then accesses length
    val l = if (b != null) b.length else -1
    print(l)
    // -1
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    // Assigns a nullable string to a variable
    val b: String? = "Kotlin"

    // Checks for nullability first and then accesses length
    if (b != null && b.length > 0) {
        print("String of length ${b.length}")
        // String of length 6
    } else {
        // Provides alternative if the condition is not met
        print("Empty string")
    }
//sampleEnd
}
```

[kotlin_036] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Safe call operator

Sumber: ht

Safe call operator

The safe call operator `?.` allows you to handle nullability safely in a shorter form. Instead of throwing an NPE,
if the object is `null`, the `?.` operator simply returns `null`:

{kotlin-runnable="true"}

The `b?.length` expression checks for nullability and returns `b.length` if `b` is non-null, or `null` otherwise. The type of this expression is `Int?`.

You can use the `?.` operator with both `var` and `val` variables in Kotlin:

* A nullable `var` can hold a `null` (for example, `var nullableValue: String? = null`) or a non-null value (for example, `var nullableValue: String? = "Kotlin"`). If it's a non-null value, you can change it to `null` at any point.
* A nullable `val` can hold a `null` (for example, `val nullableValue: String? = null`) or a non-null value (for example, `val nullableValue: String? = "Kotlin"`). If it's a non-null value, you cannot change it to `null` subsequently.

Safe calls are useful in chains. For example, Bob is an employee who may be assigned to a department (or not). That department
may, in turn, have another employee as a department head. To obtain the name of Bob's department head (if there is one),
you write the following:

This chain returns `null` if any of its properties are `null`.

You can also place a safe call on the left side of an assignment:

In the example above, if one of the receivers in the safe call chain is `null`, the assignment is skipped, and the expression on the right is not evaluated at all. For example, if either
`person` or `person.department` is `null`, the function is not called. Here's the equivalent of the same safe call but with the `if` conditional:

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Assigns a nullable string to a variable
    val a: String? = "Kotlin"
    // Assigns null to a nullable variable
    val b: String? = null

    // Checks for nullability and returns length or null
    println(a?.length)
    // 6
    println(b?.length)
    // null
//sampleEnd
}

// ----------------------------------------

bob?.department?.head?.name

// ----------------------------------------

person?.department?.head = managersPool.getManager()
```

[kotlin_037] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Elvis operator

Sumber: ht

Elvis operator

When working with nullable types, you can check for `null` and provide an alternative value. For example, if `b` is not `null`,
access `b.length`. Otherwise, return an alternative value:

{kotlin-runnable="true"}

Instead of writing the complete `if` expression, you can handle this in a more concise way with the Elvis operator `?:`:

{kotlin-runnable="true"}

If the expression to the left of `?:` is not `null`, the Elvis operator returns it. Otherwise, the Elvis operator returns the expression
to the right. The expression on the right-hand side is evaluated only if the left-hand side is `null`.

Since `throw` and `return` are expressions in Kotlin, you can also use them on
the right-hand side of the Elvis operator. This can be handy, for example, when checking function arguments:

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability. If not null, returns length. If null, returns 0
    val l: Int = if (b != null) b.length else 0
    println(l)
    // 0
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    // Assigns null to a nullable variable
    val b: String? = null
    // Checks for nullability. If not null, returns length. If null, returns a non-null value
    val l = b?.length ?: 0
    println(l)
    // 0
//sampleEnd
}

// ----------------------------------------

fun foo(node: Node): String? {
    // Checks for getParent(). If not null, it's assigned to parent. If null, returns null
    val parent = node.getParent() ?: return null
    // Checks for getName(). If not null, it's assigned to name. If null, throws exception
    val name = node.getName() ?: throw IllegalArgumentException("name expected")
    // ...
}
```

[kotlin_038] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Not-null assertion operator

Sumber: ht

Not-null assertion operator
The not-null assertion operator `!!` converts any value to a non-nullable type.
When you apply the `!!` operator to a variable whose value is not `null`, it's safely handled as a non-nullable type,
and the code executes normally. However, if the value is `null`, the `!!` operator forces it to be treated as non-nullable,
which results in an NPE.
When `b` is not `null` and the `!!` operator makes it return its non-null value (which is a `String` in this example), it accesses `length` correctly:
{kotlin-runnable="true"}
When `b` is `null` and the `!!` operator makes it return its non-null value, and an NPE occurs:
{kotlin-runnable="true" kotlin-min-compiler-version="2.0" validate="false"}
The `!!` operator is particularly useful
when you are confident that a value is not `null` and there's no chance of getting an NPE, but the compiler cannot guarantee this due to certain rules.
In such cases, you can use the `!!` operator to explicitly tell the compiler that the value is not `null`.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Assigns a nullable string to a variable
    val b: String? = "Kotlin"
    // Treats b as non-null and accesses its length
    val l = b!!.length
    println(l)
    // 6
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    // Assigns null to a nullable variable
    val b: String? = null
    // Treats b as non-null and tries to access its length
    val l = b!!.length
    println(l)
    // Exception in thread "main" java.lang.NullPointerException
//sampleEnd
}
```

[kotlin_039] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Nullable receiver

Sumber: ht

Nullable receiver
You can use extension functions with a nullable receiver type,
allowing these functions to be called on variables that might be `null`.
By defining an extension function on a nullable receiver type, you can handle `null` values within the function itself
instead of checking for `null` at every place where you call the function.
For example, the `.toString()` extension function
can be called on a nullable receiver. When invoked on a `null` value, it safely returns the string `"null"` without throwing

an exception:
{kotlin-runnable="true"}
In the example above, even though `person` is `null`, the `.toString()` function safely returns the string `"null"`. This can be helpful for debugging and logging.
If you expect the `.toString()` function to return a nullable string (either a string representation or `null`), use the safe-call operator `?.`.
The `?.` operator calls `.toString()` only if the object is not `null`, otherwise it returns `null`:
{kotlin-runnable="true"}
The `?.` operator allows you to safely handle potential `null` values while still accessing properties or functions of objects that might be `null`.

Contoh Kode / Perintah CLI:

```
//sampleStart
fun main() {
    // Assigns null to a nullable Person object stored in the person variable
    val person: Person? = null

    // Applies .toString to the nullable person variable and prints a string
    println(person.toString())
    // null
}

// Defines a simple Person class
data class Person(val name: String)
//sampleEnd

// ----------------------------------------

//sampleStart
fun main() {
    // Assigns a nullable Person object to a variable
    val person1: Person? = null
    val person2: Person? = Person("Alice")

    // Prints "null" if person is null; otherwise prints the result of person.toString()
    println(person1?.toString())
    // null
    println(person2?.toString())
    // Person(name=Alice)
}

// Defines a Person class
data class Person(val name: String)
//sampleEnd
```

[kotlin_040] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Let function

Sumber: ht

Let function
To handle `null` values and perform operations only on non-null types, you can use the safe call operator `?.` together with the
`let` function.
This combination is useful for evaluating an expression, check the result for `null`, and execute code only if it's not `null`, avoiding manual null checks:
{kotlin-runnable="true"}

Contoh Kode / Perintah CLI:

```
fun main() {
```

```
//sampleStart
    // Declares a list of nullable strings
    val listWithNulls: List<String?> = listOf("Kotlin", null)

    // Iterates over each item in the list
    for (item in listWithNulls) {
        // Checks if the item is null and only prints non-null values
        item?.let { println(it) }
        //Kotlin
    }
//sampleEnd
}
```

[kotlin_041] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Safe casts

Sumber: ht

Safe casts
The regular Kotlin operator for type casts is the `as` operator. However, regular casts can result in an exception if the object is not of the target type.
You can use the `as?` operator for safe casts. It tries to cast a value to the specified type and returns `null` if the value is not of that type:
{kotlin-runnable="true"}
The code above prints `null` because `a` is not an `Int`, so the cast fails safely. It also prints `"Hello, Kotlin!"` because it matches the `String?` type, so the safe cast succeeds.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Declares a variable of type Any, which can hold any type of value
    val a: Any = "Hello, Kotlin!"

    // Safe casts to Int using the 'as?' operator
    val aInt: Int? = a as? Int
    // Safe casts to String using the 'as?' operator
    val aString: String? = a as? String

    println(aInt)
    // null
    println(aString)
    // "Hello, Kotlin!"
//sampleEnd
}
```

[kotlin_042] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): Collections of a nullable type

Sumber: ht

Collections of a nullable type
If you have a collection of nullable elements and want to keep only the non-null ones, use the `filterNotNull()` function:
{kotlin-runnable="true"}

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // Declares a list containing some null and non-null integer values
    val nullableList: List<Int?> = listOf(1, 2, null, 4)
```

```
    // Filters out null values, resulting in a list of non-null integers
    val intList: List<Int> = nullableList.filterNotNull()

    println(intList)
    // [1, 2, 4]
//sampleEnd
}
```

[kotlin_043] Kotlin - Null Safety & Type System

Null Safety (Nullable types ?, Safe calls ?., Elvis operator ?:, Safe casts): What's next?

Sumber: ht

What's next?
* Learn how to handle nullability in Java and Kotlin.
* Learn about generic types that are definitely non-nullable.

[kotlin_044] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Overview

Sumber: ht

[//]: # (title: Classes)
> Before creating classes, consider using a data class if the purpose is to store data.
> Alternatively, think about extending an existing class with an extension, rather than creating a new one from scratch.
>
{style="tip"}
Like other object-oriented languages, Kotlin uses _classes_ to encapsulate data (properties) and behavior (functions) for reusable, structured code.
Classes are blueprints or templates for objects, which you
create via constructors.
When you create an instance of a class, you are creating
a concrete object based on that blueprint.
Kotlin offers concise syntax for declaring classes. To declare a class, use the `class` keyword
followed by the class name:
The class declaration consists of:
* **Class header**, including but not limited to:
* `class` keyword
* Class name
* Type parameters (if any)
* Primary constructor (optional)
* **Class body** (optional), surrounded by curly braces `{}`, and including **class members** such as:
* Secondary constructors
* Initializer blocks
* Functions
* Properties
* Nested and inner classes
* Object declarations
You can keep both the class header and body to a bare minimum.
If the class doesn't have a body, you can omit the curly braces `{}`:
Here's an example that declares a class with a header and body,
then creates an instance from it:
{kotlin-runnable="true" id="class-with-header-and-body"}

Contoh Kode / Perintah CLI:

```
class Person { /*...*/ }

// ----------------------------------------

// Class with primary constructor, but without a body
class Person(val name: String, var age: Int)

// ----------------------------------------

// Person class with a primary constructor
// that initializes the name property
class Person(val name: String) {
    // Class body with age property
    var age: Int = 0
}

fun main() {
    // Creates an instance of the Person class by calling the constructor
    val person = Person("Alice")

    // Accesses the instance's properties
    println(person.name)
    // Alice
    println(person.age)
    // 0
}
```

[kotlin_045] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Creating instances

Sumber: ht

Creating instances
An instance is created
when you use the class as a blueprint to build a real object to work with in your program.
To create an instance of a class, use the class name followed by parentheses `()`, similar to calling a function:
In Kotlin, you can create instances:
* **Without arguments** (`Person()`): creates an instance using the default values, if they are declared in the class.
* **With arguments** (`Person(value)`): creates an instance by passing specific values.
You can assign the created instance to a mutable (`var`) or read-only (`val`) variable:
It's possible to create instances wherever you need them, inside the `main()` function, within other functions, or inside another class.
Additionally, you can create instances inside another function and call that function from `main()`.
The following code declares a `Person` class with a property for storing a name.
It also demonstrates
how to create an instance with both the default constructor's value and a specific value:
{kotlin-runnable="true" id="create-instance-of-a-class"}
> In Kotlin, unlike other object-oriented programming languages,
> there is no need for the `new` keyword when creating class instances.
>
{style="note"}
For information about creating instances of nested, inner, and anonymous inner classes,
see the Nested classes section.

Contoh Kode / Perintah CLI:

```
// Creates an instance of the Person class
```

```
val anonymousUser = Person()

// ----------------------------------------

// Creates an instance using the default value
// and assigns it to a mutable variable
var anonymousUser = Person()

// Creates an instance by passing a specific value
// and assigns it to a read-only variable
val namedUser = Person("Joe")

// ----------------------------------------

// Class header with a primary constructor
// that initializes name with a default value
class Person(val name: String = "Sebastian")

fun main() {
    // Creates an instance using the default constructor's value
    val anonymousUser = Person()

    // Creates an instance by passing a specific value
    val namedUser = Person("Joe")

    // Accesses the instances' name property
    println(anonymousUser.name)
    // Sebastian
    println(namedUser.name)
    // Joe
}
```

[kotlin_046] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Constructors and initializer blocks

Sumber: ht

Constructors and initializer blocks
When you create a class instance, you call one of its constructors. A class in Kotlin can have a
primary constructor and one or more _secondary constructors_.
The primary constructor is the main way to initialize a class.
You declare it in the class header.
A secondary constructor provides additional initialization logic.
You declare it in the class body.
Both primary and secondary constructors are optional, but a class must have at least one constructor.

[kotlin_047] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Primary constructor

Sumber: ht

Primary constructor
The primary constructor sets up the initial state of an instance when it's created.
To declare a primary constructor, place it in the class header after the class name:
If the primary constructor doesn't have any annotations or visibility modifiers,
you can omit the `constructor` keyword:
The primary constructor can declare parameters as properties. Use the `val` keyword before the argument name to

declare a read-only property
and the `var` keyword for a mutable property:
These constructor parameter properties are stored as part of the instance and are accessible from outside the class.
It's also possible to declare primary constructor parameters that are not properties.
These parameters don't have
`val` or `var` in front of them, so they are not stored in the instance and are available only within the class body:
Properties declared in the primary constructor are accessible by
member functions of the class:
You can also assign default values to properties in the primary constructor:
If no value is passed to the constructor during instance creation,
properties use their default value:
{kotlin-runnable="true" id="class-with-primary-constructor"}
You can use the primary constructor parameters to initialize additional class properties directly in the class body:
{kotlin-runnable="true" id="class-with-default-values"}
As with functions, you can use trailing commas in constructor declarations:

Contoh Kode / Perintah CLI:

```
class Person constructor(name: String) { /*...*/ }

// ----------------------------------------

class Person(name: String) { /*...*/ }

// ----------------------------------------

class Person(val name: String, var age: Int) { /*...*/ }
```

[kotlin_048] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Initializer blocks

Sumber: ht

Initializer blocks
The primary constructor initializes the class and sets its properties.
In most cases, you can handle this with simple code.
If you need to perform more complex operations during instance creation,
place that logic in _initializer blocks_ inside the class body. These blocks run when the primary constructor executes.
Declare initializer blocks with the `init` keyword followed by curly braces `{}`.
Write within the curly braces any code that you want to run during initialization:
{kotlin-runnable="true" id="class-with-initializer-block"}
Add as many initializer blocks (`init {}`) as you need. They run in the order in which they appear in the class body,
along with property initializers:
{kotlin-runnable="true" id="class-with-second-initializer-block"}
You can use primary constructor parameters in initializer blocks. For example, in the code above, the first and second initializers use
the `name` and `age` parameters from the primary constructor.
A common use case for `init` blocks is data validation. For example, by calling the `require` function:

Contoh Kode / Perintah CLI:

```
// Class with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {
    init {
        // Initializer block runs when an instance is created
        println("Person created: $name, age $age.")
    }
}
```

```
fun main() {
    // Creates an instance of the Person class
    Person("John", 30)
    // Person created: John, age 30.
}

// ----------------------------------------

//sampleStart
// Class with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {
    // First initializer block
    init {
        // Runs first when an instance is created
        println("Person created: $name, age $age.")
    }

    // Second initializer block
    init {
        // Runs after the first initializer block
        if (age < 18) {
            println("$name is a minor.")
        } else {
            println("$name is an adult.")
        }
    }
}

fun main() {
    // Creates an instance of the Person class
    Person("John", 30)
    // Person created: John, age 30.
    // John is an adult.
}
//sampleEnd

// ----------------------------------------

class Person(val age: Int) {
    init {
        require(age > 0) { "age must be positive" }
    }
}
```

[kotlin_049] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Secondary constructors

Sumber: ht

Secondary constructors

In Kotlin, secondary constructors are additional constructors that a class can have beyond its primary constructor.
Secondary constructors are useful
when you need multiple ways to initialize a class or for Java interoperability.
To declare a secondary constructor, use the `constructor`
keyword inside the class body with the constructor parameters within parentheses `()`.
Add the constructor logic within curly braces `{}`:
{kotlin-runnable="true" id="class-with-secondary-constructor"}
> The expression `age.toIntOrNull() ?: 0` uses the Elvis operator. For more information, see Null safety.

>
{style="tip"}
In the code above, the secondary constructor delegates to the primary constructor via the `this` keyword,
passing `name` and the `age` value converted to an integer.
In Kotlin, secondary constructors must delegate to the primary constructor. This delegation ensures that all primary constructor initialization logic is executed before any secondary constructor logic runs.
Constructor delegation can be:
* **Direct**, where the secondary constructor calls the primary constructor immediately.
* **Indirect**, where one secondary constructor calls another, which in turn delegates to the primary constructor.
Here's an example demonstrating how direct and indirect delegation works:
{kotlin-runnable="true" id="class-delegation"}
In classes with initializer blocks (`init {}`), the code within these blocks becomes part of the primary constructor.
Given that secondary constructors delegate to the primary constructor first, all initializer blocks
and property initializers run before the body of the secondary constructor. Even if the class has no primary constructor, the delegation still happens implicitly:
{kotlin-runnable="true" id="class-delegation-sequen

Contoh Kode / Perintah CLI:

```
// Class header with a primary constructor that initializes name and age
class Person(val name: String, var age: Int) {

    // Secondary constructor that takes age as a
    // String and converts it to an Int
    constructor(name: String, age: String) : this(name, age.toIntOrNull() ?: 0) {
        println("$name created with converted age: ${this.age}")
    }
}

fun main() {
    // Uses the secondary constructor with age as a String
    Person("Bob", "8")
    // Bob created with converted age: 8
}

// ----------------------------------------

// Class header with a primary constructor that initializes name and age
class Person(
    val name: String,
    var age: Int
) {
    // Secondary constructor with direct delegation
    // to the primary constructor
    constructor(name: String) : this(name, 0) {
        println("Person created with default age: $age and name: $name.")
    }

    // Secondary constructor with indirect delegation:
    // this("Bob") -> constructor(name: String) -> primary constructor
    constructor() : this("Bob") {
        println("New person created with default age: $age and name: $name.")
    }
}

fun main() {
    // Creates an instance based on the direct delegation
    Person("Alice")
    // Person created with default age: 0 and name: Alice.
```

```
    // Creates an instance based on the indirect delegation
    Person()
    // Person created with default age: 0 and name: Bob.
    // New person created with default age: 0 and name: Bob.
}

// ----------------------------------------

// Class header with no primary constructor
class Person {
    // Initializer block runs when an instance is created
    init {
        // Runs before the secondary constructor
        println("1. First initializer block runs")
    }

    // Secondary constructor that takes an integer parameter
    constructor(i: Int) {
        // Runs after the initializer block
        println("2. Person $i is created")
    }
}

fun main() {
    // Creates an instance of the Person class
    Person(1)
    // 1. First initializer block runs
    // 2. Person 1 created
}
```

[kotlin_050] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Classes without constructors

Sumber: ht

Classes without constructors

Classes that don't declare any constructors (primary or secondary) have an implicit primary constructor
with no parameters:

The visibility of this implicit primary constructor is public, meaning it can be accessed from anywhere.
If you don't want your class to have a public constructor, declare an empty primary constructor with non-default visibility:

> On the JVM, if all primary constructor parameters have default values, the compiler implicitly provides
> a parameterless constructor that uses those default values.
>
> This makes it easier to use Kotlin with libraries such as Jackson
> or Spring Data JPA, which create class instances through
> parameterless constructors.
>
> In the following example, Kotlin implicitly provides a parameterless constructor `Person()` that uses the default value `""`:
>
>
>

{style="note"}

Contoh Kode / Perintah CLI:

```
// Class with no explicit constructors
class Person {
    // No primary or secondary constructors declared
```

```
}

fun main() {
    // Creates an instance of the Person class
    // using the implicit primary constructor
    val person = Person()
}

// ----------------------------------------

class Person private constructor() { /*...*/ }

// ----------------------------------------

> class Person(val personName: String = "")
>
```

[kotlin_051] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Inheritance

Sumber: ht

Inheritance
Class inheritance in Kotlin allows you to create a new class (derived class) from an existing class (base class),
inheriting its properties and functions while adding or modifying behavior.
For detailed information about inheritance hierarchies and how to use the `open` keyword, see the Inheritance section.

[kotlin_052] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Abstract classes

Sumber: ht

Abstract classes
In Kotlin, abstract classes are classes that can't be instantiated directly. They are designed to be inherited by other
classes which define their actual behavior. This behavior is called an _implementation_.
An abstract class can declare abstract properties and functions, which must be implemented
by subclasses.
Abstract classes can also have constructors.
These constructors initialize class properties and enforce required parameters for subclasses.
Declare an abstract class using the `abstract` keyword:
An abstract class can have both abstract and non-abstract members (properties and functions).
To declare a member as abstract, you must use the `abstract` keyword explicitly.
You don't need to annotate abstract classes or functions with the `open`
keyword because they are implicitly inheritable by default.
For more details about the `open` keyword, see Inheritance.
Abstract members don't have an implementation
in the abstract class.
You define the implementation in a subclass or inheriting class with an `override` function or property:
{kotlin-runnable="true" id="abstract-class"}

Contoh Kode / Perintah CLI:

```
abstract class Person(val name: String, val age: Int)

// ----------------------------------------

// Abstract class with a primary constructor that declares name and age
```

```
abstract class Person(
    val name: String,
    val age: Int
) {
    // Abstract member
    // Doesn't provide implementation,
    // and it must be implemented by subclasses
    abstract fun introduce()

    // Non-abstract member (has an implementation)
    fun greet() {
        println("Hello, my name is $name.")
    }
}

// Subclass that provides an implementation for the abstract member
class Student(
    name: String,
    age: Int,
    val school: String
) : Person(name, age) {
    override fun introduce() {
        println("I am $name, $age years old, and I study at $school.")
    }
}

fun main() {
    // Creates an instance of the Student class
    val student = Student("Alice", 20, "Engineering University")

    // Calls the non-abstract member
    student.greet()
    // Hello, my name is Alice.

    // Calls the overridden abstract member
    student.introduce()
    // I am Alice, 20 years old, and I study at Engineering University.
}
```

[kotlin_053] Kotlin - Classes & Modern OOP

Classes and Inheritance (Primary/Secondary Constructors, Init blocks, Abstract classes): Companion objects

Sumber: ht

Companion objects
In Kotlin, each class can have a companion object.
Companion objects are a type of object declaration
that allows you to access its members using the class name without creating a class instance.
Suppose you need to write a function that can be called without creating an instance of a class, but it is still logically connected to the class (such as a factory function). In that case, you can declare it inside a companion object declaration within the class:
{kotlin-runnable="true" id="class-with-companion-object"}
If you declare a companion object inside your class,
you can access its members using only the class name as a qualifier.
For more information, see Companion objects.

Contoh Kode / Perintah CLI:

```
// Class with a primary constructor that declares the name property
class Person(
```

```
    val name: String
) {
    // Class body with a companion object
    companion object {
        fun createAnonymous() = Person("Anonymous")
    }
}

fun main() {
    // Calls the function without creating an instance of the class
    val anonymous = Person.createAnonymous()
    println(anonymous.name)
    // Anonymous
}
```

[kotlin_054] Kotlin - Modern Data Carriers - Data Classes

Data Classes (copy(), Destructuring declarations, componentN() functions): Overview

Sumber: ht

[//]: # (title: Data classes)
Data classes in Kotlin are primarily used to hold data. For each data class, the compiler automatically generates additional member functions that allow you to print an instance to readable output, compare instances, copy instances, and more.
Data classes are marked with `data`:
The compiler automatically derives the following members from all properties declared in the primary constructor:
* `equals()`/`hashCode()` pair.
* `toString()` of the form `"User(name=John, age=42)"`.
* `componentN()` functions corresponding to the properties in their order of declaration.
* `copy()` function.
To ensure consistency and meaningful behavior of the generated code, data classes have to fulfill the following requirements:
* The primary constructor must have at least one parameter.
* All primary constructor parameters must be marked as `val` or `var`.
* Data classes can't be abstract, open, sealed, or inner.
Additionally, the generation of data class members follows these rules with regard to the members' inheritance:
* If there are explicit implementations of `equals()`, `hashCode()`, or `toString()` in the data class body or `final` implementations in a superclass, then these functions are not generated, and the existing implementations are used.
* If a supertype has `componentN()` functions that are `open` and return compatible types, the corresponding functions are generated for the data class and override those of the supertype. If the functions of the supertype cannot be overridden due to incompatible signatures or due to their being final, an error is reported.
* Providing explicit implementations for the `componentN()` and `copy()` functions is not allowed.
Data classes may extend other classes (see Sealed classes for examples).
> On the JVM, if

Contoh Kode / Perintah CLI:

```
data class User(val name: String, val age: Int)

// ----------------------------------------

> data class User(val name: String = "", val age: Int = 0)
>
```

[kotlin_055] Kotlin - Modern Data Carriers - Data Classes

Data Classes (copy(), Destructuring declarations, componentN() functions): Properties declared in the class body

Sumber: ht

Properties declared in the class body
The compiler only uses the properties defined inside the primary constructor for the automatically generated functions. To exclude a property from the generated implementations, declare it inside the class body:
In the example below, only the `name` property is used by default inside the `toString()`, `equals()`, `hashCode()`, and `copy()` implementations, and there is only one component function, `component1()`.
The `age` property is declared inside the class body and is excluded.
Therefore, two `Person` objects with the same `name` but different `age` values are considered equal since `equals()` only evaluates properties from the primary constructor:
{kotlin-runnable="true"}

Contoh Kode / Perintah CLI:

```
data class Person(val name: String) {
    var age: Int = 0
}

// ----------------------------------------

data class Person(val name: String) {
    var age: Int = 0
}
fun main() {
//sampleStart
    val person1 = Person("John")
    val person2 = Person("John")
    person1.age = 10
    person2.age = 20

    println("person1 == person2: ${person1 == person2}")
    // person1 == person2: true

    println("person1 with age ${person1.age}: ${person1}")
    // person1 with age 10: Person(name=John)

    println("person2 with age ${person2.age}: ${person2}")
    // person2 with age 20: Person(name=John)
//sampleEnd
}
```

[kotlin_056] Kotlin - Modern Data Carriers - Data Classes

Data Classes (copy(), Destructuring declarations, componentN() functions): Copying

Sumber: ht

Copying
Use the `copy()` function to copy an object, allowing you to alter _some_ of its properties while keeping the rest unchanged.
The implementation of this function for the `User` class above would be as follows:
You can then write the following:
The `copy()` function creates a _shallow_ copy of the instance. In other words, it doesn't copy components recursively. As a result, references to other objects are shared.
For example, if a property holds a mutable list, changes made through the "original" value are also visible through the copy,
and changes made through the copy are visible through the original:

{kotlin-runnable="true"}

As you can see, modifying the `duplicate.roles` property also changes the `original.roles` property because both properties share the same list reference.

Contoh Kode / Perintah CLI:

```
fun copy(name: String = this.name, age: Int = this.age) = User(name, age)

// ----------------------------------------

val jack = User(name = "Jack", age = 1)
val olderJack = jack.copy(age = 2)

// ----------------------------------------

data class Employee(val name: String, val roles: MutableList<String>)

fun main() {
    val original = Employee("Jamie", mutableListOf("developer"))
    val duplicate = original.copy()

    duplicate.roles.add("team lead")

    println(original)
    // Employee(name=Jamie, roles=[developer, team lead])
    println(duplicate)
    // Employee(name=Jamie, roles=[developer, team lead])
}
```

[kotlin_057] Kotlin - Modern Data Carriers - Data Classes

Data Classes (copy(), Destructuring declarations, componentN() functions): Data classes and destructuring declarations

Sumber: ht

Data classes and destructuring declarations

Component functions generated for data classes make it possible to use them in destructuring declarations:

Contoh Kode / Perintah CLI:

```
val jane = User("Jane", 35)
val (name, age) = jane
println("$name, $age years of age")
// Jane, 35 years of age
```

[kotlin_058] Kotlin - Modern Data Carriers - Data Classes

Data Classes (copy(), Destructuring declarations, componentN() functions): Standard data classes

Sumber: ht

Standard data classes

The standard library provides the `Pair` and `Triple` classes. In most cases, though, named data classes are a better design choice

because they make the code easier to read by providing meaningful names for the properties.

[kotlin_059] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Overview

Sumber: ht

[//]: # (title: Sealed classes and interfaces)
Sealed classes and interfaces provide controlled inheritance of your class hierarchies.
All direct subclasses of a sealed class are known at compile time. No other subclasses may appear outside the module and
package within which the sealed class is defined. The same logic applies to sealed interfaces and their implementations:
once a module with a sealed interface is compiled, no new implementations can be created.
> Direct subclasses are classes that immediately inherit from their superclass.
>
> Indirect subclasses are classes that inherit from more than one level down from their superclass.
>
{style="note"}
When you combine sealed classes and interfaces with the `when` expression, you can cover the behavior of all possible subclasses and ensure that no new subclasses are created to affect your code adversely.
Sealed classes are best used for scenarios when:
* **Limited class inheritance is desired:** You have a predefined, finite set of subclasses that extend a class, all of which are known at compile time.
* **Type-safe design is required:** Safety and pattern matching are crucial in your project. Particularly for state management or handling complex conditional logic. For an example, check out Use sealed classes with when expressions.
* **Working with closed APIs:** You want robust and maintainable public APIs for libraries that ensure that third-party clients use the APIs as intended.
For more detailed practical applications, see Use case scenarios.
> Java 15 introduced a similar concept,
> where sealed classes use the `sealed` keyword along with the `permits` clause to define restricted hierarchies.
>
{style="tip"}

[kotlin_060] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Declare a sealed class or interface

Sumber: ht

Declare a sealed class or interface
To declare a sealed class or interface, use the `sealed` modifier:
This example could represent a library's API that contains error classes to let library users handle errors that it can throw.
If the hierarchy of such error classes includes interfaces or abstract classes visible in the public API, then nothing prevents other developers from implementing or extending them in the client code.
Since the library doesn't know about errors declared outside of it, it can't treat them consistently with its own classes.
However, with a **sealed** hierarchy of error classes, library authors can be sure that they know all the possible error types and that other error types can't appear later.
The hierarchy of the example looks like this:
!Hierarchy illustration of sealed classes and interfaces{width=700}

Contoh Kode / Perintah CLI:

```
// Create a sealed interface
sealed interface Error
```

```
// Create a sealed class that implements sealed interface Error
sealed class IOError(): Error

// Define subclasses that extend sealed class 'IOError'
class FileReadError(val file: File): IOError()
class DatabaseError(val source: DataSource): IOError()

// Create a singleton object implementing the 'Error' sealed interface
object RuntimeError : Error
```

[kotlin_061] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Constructors

Sumber: ht

Constructors

A sealed class itself is always an abstract class, and as a result, can't be instantiated directly.

However, it may contain or inherit constructors. These constructors aren't for creating instances of the sealed class itself but for its subclasses. Consider the following example with a sealed class called `Error` and its several subclasses, which we instantiate:

{kotlin-runnable="true" kotlin-min-compiler-version="1.5"}

You can use `enum` classes within your sealed classes to use enum constants to represent states and provide additional detail. Each enum constant exists only as a **single** instance, while subclasses of a sealed class may have **multiple** instances.

In the example, the `sealed class Error` along with its several subclasses, employs an `enum` to denote error severity. Each subclass constructor initializes the `severity` and can alter its state:

Constructors of sealed classes can have one of two visibilities: `protected` (by default) or `private`:

Contoh Kode / Perintah CLI:

```
sealed class Error(val message: String) {
    class NetworkError : Error("Network failure")
    class DatabaseError : Error("Database cannot be reached")
    class UnknownError : Error("An unknown error has occurred")
}

fun main() {
    val errors = listOf(Error.NetworkError(), Error.DatabaseError(), Error.UnknownError())
    errors.forEach { println(it.message) }
}
// Network failure
// Database cannot be reached
// An unknown error has occurred

// ----------------------------------------

enum class ErrorSeverity { MINOR, MAJOR, CRITICAL }

sealed class Error(val severity: ErrorSeverity) {
    class FileReadError(val file: File): Error(ErrorSeverity.MAJOR)
    class DatabaseError(val source: DataSource): Error(ErrorSeverity.CRITICAL)
    object RuntimeError : Error(ErrorSeverity.CRITICAL)
    // Additional error types can be added here
}

// ----------------------------------------

sealed class IOError {
    // A sealed class constructor has protected visibility by default. It's visible inside this class and
```

```
its subclasses
    constructor() { /*...*/ }

    // Private constructor, visible inside this class only.
    // Using a private constructor in a sealed class allows for even stricter control over instantiation,
enabling specific initialization procedures within the class.
    private constructor(description: String): this() { /*...*/ }

     // This will raise an error because public and internal constructors are not allowed in sealed
classes
    // public constructor(code: Int): this() {}
}
```

[kotlin_062] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Inheritance

Sumber: ht

Inheritance
Direct subclasses of sealed classes and interfaces must be declared in the same package. They may be top-level or nested
inside any number of other named classes, named interfaces, or named objects. Subclasses can have any visibility
as long as they are compatible with normal inheritance rules in Kotlin, including those for overriding properties.
Subclasses of sealed classes must have a properly qualified name. They can't be local or anonymous objects.
> `enum` classes can't extend a sealed class, or any other class. However, they can implement sealed interfaces:
>
>
>
{style="note"}
These restrictions don't apply to indirect subclasses. If a direct subclass of a sealed class is not marked as sealed,
it can be extended in any way that its modifiers allow:

Contoh Kode / Perintah CLI:

```
> sealed interface Error
>
> // enum class extending the sealed interface Error
> enum class ErrorType : Error {
>     FILE_ERROR, DATABASE_ERROR
> }
>
>

// ----------------------------------------

// Sealed interface 'Error' has implementations only in the same package and module
sealed interface Error

// Sealed class 'IOError' extends 'Error' and is extendable only within the same package
sealed class IOError(): Error

// Open class 'CustomError' extends 'Error' and can be extended anywhere it's visible
open class CustomError(): Error
```

[kotlin_063] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Inheritance in multiplatform projects

Sumber: ht

Inheritance in multiplatform projects
There is one more inheritance restriction in multiplatform projects: direct subclasses of sealed classes must
reside in the same source set. It applies to sealed classes without the expected and actual modifiers.
If a sealed class is declared as `expect` in a common source set and have `actual` implementations in platform source sets,
both `expect` and `actual` versions can have subclasses in their source sets. Moreover, if you use a hierarchical structure,
you can create subclasses in any source set between the `expect` and `actual` declarations.
Learn more about the hierarchical structure of multiplatform projects.

[kotlin_064] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Use sealed classes with when expression

Sumber: ht

Use sealed classes with when expression
The key benefit of using sealed classes comes into play when you use them in a `when`
expression.
The `when` expression, used with a sealed class, allows the Kotlin compiler to check exhaustively that all possible cases are covered.
In such cases, you don't need to add an `else` clause:
{kotlin-runnable="true" kotlin-min-compiler-version="1.5"}
> To reduce repetition in `when` expressions, try out context-sensitive resolution (currently in preview).
> This feature allows you to omit the type name when matching sealed class members if the expected type is known.
>
> For more information, see Preview of context-sensitive resolution or the related KEEP proposal.
>
{style="tip"}
When using sealed classes with `when` expressions, you can also add guard conditions to include additional checks in a single branch.
For more information, see Guard conditions in when expressions.
> In multiplatform projects, if you have a sealed class with a `when` expression as an
> expected declaration in your common code, you still need an `else` branch.
> This is because subclasses of `actual` platform implementations may extend sealed classes that
> aren't known in the common code.
>
{style="note"}

Contoh Kode / Perintah CLI:

```
// Sealed class and its subclasses
sealed class Error {
    class FileReadError(val file: String): Error()
    class DatabaseError(val source: String): Error()
    object RuntimeError : Error()
}

//sampleStart
// Function to log errors
fun log(e: Error) = when(e) {
    is Error.FileReadError -> println("Error while reading file ${e.file}")
```

```
    is Error.DatabaseError -> println("Error while reading from database ${e.source}")
    Error.RuntimeError -> println("Runtime error")
    // No `else` clause is required because all the cases are covered
}
//sampleEnd

// List all errors
fun main() {
    val errors = listOf(
        Error.FileReadError("example.txt"),
        Error.DatabaseError("usersDatabase"),
        Error.RuntimeError
    )

    errors.forEach { log(it) }
}
```

[kotlin_065] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Inspect sealed subclasses with reflection

Sumber: ht

Inspect sealed subclasses with reflection
You can use Reflection functionality to inspect the direct subclasses of a sealed class or interface at runtime.
It helps a framework code discover the variants of a sealed hierarchy. For example, you can register a
serializer or verify that every variant has a handler.
If you need to inspect a known direct subclass, use sealedSubclasses.
The method returns a `KClass` that can represent a regular class or an object declaration. For an object declaration,
`objectInstance` returns its singleton instance:
In this example, `objectInstance` filters out `Send`, because every `Send` value must be constructed separately. Since `Start`
and `Stop` are object declarations, the compiler returns their existing singleton instances.
> If one of the direct subclasses is also sealed, traverse its `sealedSubclasses` recursively to collect the complete hierarchy.
>
{style="tip"}

Contoh Kode / Perintah CLI:

```
sealed interface Status
data class Send(val message: String) : Status
data object Start : Status
data object Stop : Status

fun main() {
    // Get the direct subclasses known to the sealed hierarchy
    val subclasses = Status::class.sealedSubclasses
    println(subclasses.mapNotNull { it.simpleName })
    // [Send, Start, Stop]

    val singletonStatus = subclasses.mapNotNull { subclass ->
        // Return null for Send because it is a regular class
        subclass.objectInstance
    }
    println(singletonStatus)
    // [Start, Stop]
}
```

[kotlin_066] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Use case scenarios

Sumber: ht

Use case scenarios
Let's explore some practical scenarios where sealed classes and interfaces can be particularly useful.

[kotlin_067] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): State management in UI applications

Sumber: ht

State management in UI applications
You can use sealed classes to represent different UI states in an application.
This approach allows for structured and safe handling of UI changes.
This example demonstrates how to manage various UI states:

Contoh Kode / Perintah CLI:

```
sealed class UIState {
    data object Loading : UIState()
    data class Success(val data: String) : UIState()
    data class Error(val exception: Exception) : UIState()
}

fun updateUI(state: UIState) {
    when (state) {
        is UIState.Loading -> showLoadingIndicator()
        is UIState.Success -> showData(state.data)
        is UIState.Error -> showError(state.exception)
    }
}
```

[kotlin_068] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): Payment method handling

Sumber: ht

Payment method handling
In practical business applications, handling various payment methods efficiently is a common requirement.
You can use sealed classes with `when` expressions to implement such business logic.
By representing different payment methods as subclasses of a sealed class, it establishes a clear and manageable structure for processing transactions:
`Payment` is a sealed class that represents different payment methods in an e-commerce system:
`CreditCard`, `PayPal`, and `Cash`. Each subclass can have its specific properties, like `number` and `expiryDate` for `CreditCard`, and `email` for `PayPal`.
The `processPayment()` function demonstrates how to handle different payment methods.
This approach ensures that all possible payment types are considered, and the system remains flexible for new payment
methods to be added in the future.

Contoh Kode / Perintah CLI:

```
sealed class Payment {
    data class CreditCard(val number: String, val expiryDate: String) : Payment()
    data class PayPal(val email: String) : Payment()
    data object Cash : Payment()
}
```

```
fun processPayment(payment: Payment) {
    when (payment) {
        is Payment.CreditCard -> processCreditCardPayment(payment.number, payment.expiryDate)
        is Payment.PayPal -> processPayPalPayment(payment.email)
        is Payment.Cash -> processCashPayment()
    }
}
```

[kotlin_069] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): API request-response handling

Sumber: ht

API request-response handling

You can use sealed classes and sealed interfaces to implement a user authentication system that handles API requests and responses.

The user authentication system has login and logout functionalities.

The `ApiRequest` sealed interface defines specific request types: `LoginRequest` for login, and `LogoutRequest` for logout operations.

The sealed class, `ApiResponse`, encapsulates different response scenarios: `UserSuccess` with user data, `UserNotFound`

for absent users, and `Error` for any failures. The `handleRequest` function processes these requests in a type-safe manner

using a `when` expression, while `getUserById` simulates user retrieval:

Contoh Kode / Perintah CLI:

```
// Import necessary modules
import io.ktor.server.application.*
import io.ktor.server.resources.*

import kotlinx.serialization.*

// Define the sealed interface for API requests using Ktor resources
@Resource("api")
sealed interface ApiRequest

@Serializable
@Resource("login")
data class LoginRequest(val username: String, val password: String) : ApiRequest


@Serializable
@Resource("logout")
object LogoutRequest : ApiRequest

// Define the ApiResponse sealed class with detailed response types
sealed class ApiResponse {
    data class UserSuccess(val user: UserData) : ApiResponse()
    data object UserNotFound : ApiResponse()
    data class Error(val message: String) : ApiResponse()
}

// User data class to be used in the success response
data class UserData(val userId: String, val name: String, val email: String)

// Function to validate user credentials (for demonstration purposes)
fun isValidUser(username: String, password: String): Boolean {
    // Some validation logic (this is just a placeholder)
```

```
    return username == "validUser" && password == "validPass"
}

// Function to handle API requests with detailed responses
fun handleRequest(request: ApiRequest): ApiResponse {
    return when (request) {
        is LoginRequest -> {
            if (isValidUser(request.username, request.password)) {
                ApiResponse.UserSuccess(UserData("userId", "userName", "userEmail"))
            } else {
                ApiResponse.Error("Invalid username or password")
            }
        }
        is LogoutRequest -> {
            // Assuming logout operation always succeeds for this example
            ApiResponse.UserSuccess(UserData("userId", "userName", "userEmail")) // For demonstration
        }
    }
}

// Function to simulate a getUserById call
fun getUserById(userId: String): ApiResponse {
    return if (userId == "validUserId") {
        ApiResponse.UserSuccess(UserData("validUserId", "John Doe", "john@example.com"))
    } else {
        ApiResponse.UserNotFound
    }
    // Error handling would also result in an Error response.
}

// Main function to demonstrate the usage
fun main() {
    val loginResponse = handleRequest(LoginRequest("user", "pass"))
    println(loginResponse)

    val logoutResponse = handleRequest(LogoutRequest)
    println(logoutResponse)

    val userResponse = getUserById("validUserId")
    println(userResponse)

    val userNotFoundResponse = getUserById("invalidId")
    println(userNotFoundResponse)
}
```

[kotlin_070] Kotlin - Hierarchies - Sealed Classes & Interfaces

Sealed Classes and Interfaces (Restricted class hierarchies, Exhaustive when): What's next

Sumber: ht

What's next

Learn more about inheritance in Kotlin.

[kotlin_071] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Overview

Sumber: ht

[//]: # (title: Extensions)
Kotlin _extensions_ let you extend a class or an interface with new functionality without using inheritance or design patterns like _Decorator_. They are useful when working with third-party libraries you can't modify directly. Once created, you call these extensions as if they were members of the original class or interface.
The most common forms of extensions are _extension functions_ and _extension properties_.
Importantly, extensions don't modify the classes or interfaces they extend. When you define an extension, you don't add new members.
You make new functions callable or new properties accessible using the same syntax.

[kotlin_072] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Receivers

Sumber: ht

Receivers
Extensions are always called on a receiver. The receiver has to have the same type as the class or interface being extended.
To use an extension, prefix it with the receiver followed by a `.` and the function or property name.
For example, the `.appendLine()` extension function from the standard library extends the `StringBuilder` class.
So in this case, the receiver is a `StringBuilder` instance, and the _receiver type_ is `StringBuilder`:
{kotlin-runnable="true" id="kotlin-extension-function-stringbuilder"}

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    // builder is an instance of StringBuilder
    val builder = StringBuilder()
        // Calls .appendLine() extension function on builder
        .appendLine("Hello")
        .appendLine()
        .appendLine("World")
    println(builder.toString())
    // Hello
    //
    // World
}
//sampleEnd
```

[kotlin_073] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Extension functions

Sumber: ht

Extension functions
Before creating your own extension functions, see if what you are looking for is already available in the Kotlin standard library.
The standard library provides many useful extension functions for:
* Operating on collections: `.map()`, `.filter()`, `.reduce()`, `.fold()`, `.groupBy()`.
* Converting to strings: `.joinToString()`.
* Working with null values: `.filterNotNull()`.
To create your own extension function, prefix its name with a receiver type followed by a `.`. In this example, the `.truncate()`
function extends the `String` class, so the receiver type is `String`:

{kotlin-runnable="true" id="kotlin-extension-function-truncate"}

The `.truncate()` function truncates any string that it's called on by the number in the `maxLength` argument and adds an ellipsis `...`.

If the string is shorter than `maxLength`, the function returns the original string.

In this example, the `.displayInfo()` function extends the `User` interface:

{kotlin-runnable="true" id="kotlin-extension-function-interface"}

The `.displayInfo()` function returns a string containing the `name` and `email` of a `RegularUser` instance. Defining an extension on an interface like this is useful when you want to add functionality to all types that implement an interface only once.

In this example, the `.mostVoted()` function extends the `Map` class:

{kotlin-runnable="true" id="kotlin-extension-function-mostvoted"}

The `.mostVoted()` function iterates through the key-value pairs of the map it's called on and uses the `maxByOrNull()` function to return the key of the pair containing the highest value. If the map is empty, the `maxByOrNull()` function returns `null`. The `mostVoted()` function uses a safe call `?.` to only access the `key` property when the `maxByOrNull()` function

ret

Contoh Kode / Perintah CLI:

```
fun String.truncate(maxLength: Int): String {
    return if (this.length <= maxLength) this else take(maxLength - 3) + "..."
}

fun main() {
    val shortUsername = "KotlinFan42"
    val longUsername = "JetBrainsLoverForever"

    println("Short username: ${shortUsername.truncate(15)}")
    // KotlinFan42
    println("Long username:  ${longUsername.truncate(15)}")
    // JetBrainsLov...
}

// ----------------------------------------

interface User {
    val name: String
    val email: String
}

fun User.displayInfo(): String = "User(name=$name, email=$email)"

// Inherits from and implements the properties of the User interface
class RegularUser(override val name: String, override val email: String) : User

fun main() {
    val user = RegularUser("Alice", "alice@example.com")
    println(user.displayInfo())
    // User(name=Alice, email=alice@example.com)
}

// ----------------------------------------

fun Map<String, Int>.mostVoted(): String? {
    return maxByOrNull { (key, value) -> value }?.key
}

fun main() {
    val poll = mapOf(
        "Cats" to 37,
```

```
        "Dogs" to 58,
        "Birds" to 22
    )

    println("Top choice: ${poll.mostVoted()}")
    // Dogs
}
```

[kotlin_074] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Generic extension functions

Sumber: ht

Generic extension functions

To create generic extension functions, declare the generic type parameter before the function name to make it available in the receiver type expression. In this example, the `.endpoints()` function extends `List` where `T` can be any type:

{kotlin-runnable="true" id="kotlin-extension-function-endpoints"}

The `.endpoints()` function returns a pair containing the first and last elements of the list that it's called on. Inside the function body, it calls the `first()` and `last()` functions and combines their returned values into a `Pair` using the `to` infix function.

For more information about generics, see generic functions.

Contoh Kode / Perintah CLI:

```
fun <T> List<T>.endpoints(): Pair<T, T> {
    return first() to last()
}

fun main() {
    val cities = listOf("Paris", "London", "Berlin", "Prague")
    val temperatures = listOf(21.0, 19.5, 22.3)

    val cityEndpoints = cities.endpoints()
    val tempEndpoints = temperatures.endpoints()

    println("First and last cities: $cityEndpoints")
    // (Paris, Prague)
    println("First and last temperatures: $tempEndpoints")
    // (21.0, 22.3)
}
```

[kotlin_075] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Nullable receivers

Sumber: ht

Nullable receivers

You can define extension functions with a nullable receiver type, which allows you to call them on a variable even if its value is null. When the receiver is `null`, `this` is also `null`. Make sure to handle nullability correctly within your functions. For example, use `this == null` checks inside function bodies, safe calls `?.`, or the Elvis operator `?:`.

In this example, you can call the `.toString()` function without checking for `null` because the check already happens inside
the extension function:

{kotlin-runnable="true" id="kotlin-extension-function-nullable-receiver"}

Contoh Kode / Perintah CLI:

```
fun main() {
    //sampleStart
    // Extension function on nullable Any
    fun Any?.toString(): String {
        if (this == null) return "null"
        // After null check, `this` is smart-cast to non-nullable Any
        // So this call resolves to the regular toString() function
        return toString()
    }

    val number: Int? = 42
    val nothing: Any? = null

    println(number.toString())
    // 42
    println(nothing.toString())
    // null
    //sampleEnd
}
```

[kotlin_076] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Extension or member functions?

Sumber: ht

Extension or member functions?

Since extension and member function calls have the same notation, how does the compiler know which one to use?
Extension functions are dispatched _statically_, meaning the compiler determines which function to call based on the receiver type at compile time. For example:
{kotlin-runnable="true" id="kotlin-extension-function-shape"}
In this example, the compiler calls the `Shape.getName()` extension function because the parameter `shape` is declared
as type `Shape`. Because extension functions are resolved statically, the compiler chooses the function based on the declared
type, not the actual instance.
So even though the example passes a `Rectangle` instance, the `.getName()` function resolves to `Shape.getName()` since the
variable is declared as type `Shape`.
If a class has a member function and there's an extension function with the same receiver type,
the same name, and compatible arguments, the member function takes precedence. For example:
{kotlin-runnable="true" id="kotlin-extension-function-member-function"}
However, extension functions can overload member functions that have the same name but a _different_ signature:
{kotlin-runnable="true" id="kotlin-extension-function-member-function-overload"}
In this example, since an `Int` is passed to the `.printFunctionType()` function, the compiler chooses the extension function because it matches the signature. The compiler ignores the member function, which takes no arguments.

Contoh Kode / Perintah CLI:

```
fun main() {
//sampleStart
    open class Shape
    class Rectangle: Shape()

    fun Shape.getName() = "Shape"
    fun Rectangle.getName() = "Rectangle"

    fun printClassName(shape: Shape) {
        println(shape.getName())
```

```
    }

    printClassName(Rectangle())
    // Shape
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    class Example {
        fun printFunctionType() { println("Member function") }
    }

    fun Example.printFunctionType() { println("Extension function") }

    Example().printFunctionType()
    // Member function
//sampleEnd
}

// ----------------------------------------

fun main() {
//sampleStart
    class Example {
        fun printFunctionType() { println("Member function") }
    }

    // Same name but different signature
    fun Example.printFunctionType(index: Int) { println("Extension function #$index") }

    Example().printFunctionType(1)
    // Extension function #1
//sampleEnd
}
```

[kotlin_077] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Anonymous extension functions

Sumber: ht

Anonymous extension functions

You can define extension functions without giving them a name. This is useful when you want to avoid cluttering the global
namespace or when you need to pass some extension behavior as a parameter.

For example, suppose you want to extend a data class with a one-time function to calculate shipping, without giving it a name:

{kotlin-runnable="true" id="kotlin-extension-function-anonymous"}

To pass extension behavior as a parameter, use a lambda expression with a type annotation.

For example, let's say you want to check if a number is within a range without defining a named function:

{kotlin-runnable="true" id="kotlin-extension-function-anonymous-lambda"}

In this example, the `isInRange` variable holds a function of type `Int.(min: Int, max: Int) -> Boolean`. The type is an extension function on the `Int` class that takes `min` and `max` parameters and returns a `Boolean`.

The lambda body `{ min, max -> this in min..max }` checks whether the `Int` value the function is called on falls within the
range between `min` and `max` parameters. If the check is successful, the lambda returns `true`.

For more information, see Lambda expressions and anonymous functions.

Contoh Kode / Perintah CLI:

```
fun main() {
    //sampleStart
    data class Order(val weight: Double)
    val calculateShipping = fun Order.(rate: Double): Double = this.weight * rate

    val order = Order(2.5)
    val cost = order.calculateShipping(3.0)
    println("Shipping cost: $cost")
    // Shipping cost: 7.5
}

// ----------------------------------------

fun main() {
    val isInRange: Int.(min: Int, max: Int) -> Boolean = { min, max -> this in min..max }

    println(5.isInRange(1, 10))
    // true
    println(20.isInRange(1, 10))
    // false
}
```

[kotlin_078] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Extension properties

Sumber: ht

Extension properties

Kotlin supports extension properties, which are useful for performing data transformations or creating UI display helpers without cluttering the class you're working with.

To create an extension property, write the name of the class that you want to extend, followed by a `.` and the name of your property.

For example, suppose you have a data class that represents a user with a first and last name, and you want to create a property that returns an email-style username when accessed. Your code might look like this:

{kotlin-runnable="true" id="kotlin-extension-function-property"}

Since extensions don't actually add members to classes, there's no efficient way for an extension property to have a backing field. That's why initializers are not allowed for extension properties. You can define their behavior only by explicitly providing getters and setters. For example:

{kotlin-runnable="true" id="kotlin-extension-function-property-error"}

In this example, the getter uses the Elvis operator to return the house number if it exists in the `houseNumbers` map or `1`. To learn more about how to write getters and setters, see Custom getters and setters.

Contoh Kode / Perintah CLI:

```
data class User(val firstName: String, val lastName: String)

// An extension property to get a username-style email handle
val User.emailUsername: String
    get() = "${firstName.lowercase()}.${lastName.lowercase()}"

fun main() {
    val user = User("Mickey", "Mouse")
    // Calls extension property
    println("Generated email username: ${user.emailUsername}")
    // Generated email username: mickey.mouse
}
```

```
// ----------------------------------------

data class House(val streetName: String)

// Doesn't compile because there is no getter and setter
// var House.number = 1
// Error: Initializers are not allowed for extension properties

// Compiles successfully
val houseNumbers = mutableMapOf<House, Int>()
var House.number: Int
    get() = houseNumbers[this] ?: 1
    set(value) {
        println("Setting house number for ${this.streetName} to $value")
        houseNumbers[this] = value
    }

fun main() {
    val house = House("Maple Street")

    // Shows the default
    println("Default number: ${house.number} ${house.streetName}")
    // Default number: 1 Maple Street

    house.number = 99
    // Setting house number for Maple Street to 99

    // Shows the updated number
    println("Updated number: ${house.number} ${house.streetName}")
    // Updated number: 99 Maple Street
}
```

[kotlin_079] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Companion object extensions

Sumber: ht

Companion object extensions
If a class defines a companion object, you can also define extension
functions and properties for the companion object. Just like regular members of the companion object,
you can call them using only the class name as the qualifier. The compiler names the companion object `Companion` by
default:
{kotlin-runnable="true" id="kotlin-extension-function-companion-object"}

Contoh Kode / Perintah CLI:

```
class Logger {
    companion object { }
}

fun Logger.Companion.logStartupMessage() {
    println("Application started.")
}

fun main() {
    Logger.logStartupMessage()
    // Application started.
}
```

[kotlin_080] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Declaring extensions as members

Sumber: ht

Declaring extensions as members

You can declare extensions for one class inside another. Extensions like this have multiple _implicit receivers_.

An implicit receiver is an object whose members you can access without qualifying them with `this`:

* The class where you declare the extension is the _dispatch receiver_.
* The extension function's receiver type is the _extension receiver_.

Consider this example where the `Connection` class has an extension function for the `Host` class called `printConnectionString()`:

{kotlin-runnable="true" id="kotlin-extension-function-members"}

This example declares the `printConnectionString()` function inside the `Connection` class, so the `Connection` class is the

dispatch receiver. The extension function's receiver type is the `Host` class, so the `Host` class is the extension receiver.

If the dispatch receiver and the extension receiver have members with the same name, the extension receiver's member takes

precedence. To access the dispatch receiver explicitly, use the qualified `this` syntax:

Contoh Kode / Perintah CLI:

```
class Host(val hostname: String) {
    fun printHostname() { print(hostname) }
}

class Connection(val host: Host, val port: Int) {
    fun printPort() { print(port) }

    // Host is the extension receiver
    fun Host.printConnectionString() {
        // Calls Host.printHostname()
        printHostname()
        print(":")
        // Calls Connection.printPort()
        // Connection is the dispatch receiver
        printPort()
    }

    fun connect() {
        /*...*/
        // Calls the extension function
        host.printConnectionString()
    }
}

fun main() {
    Connection(Host("kotl.in"), 443).connect()
    // kotl.in:443

    // Triggers an error because the extension function isn't available outside Connection
    // Host("kotl.in").printConnectionString()
    // Unresolved reference 'printConnectionString'.
}

// ----------------------------------------

class Connection {
    fun Host.getConnectionString() {
```

```
        // Calls Host.toString()
        toString()
        // Calls Connection.toString()
        this@Connection.toString()
    }
}
```

[kotlin_081] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Overriding member extensions

Sumber: ht

Overriding member extensions

You can declare member extensions as `open` and override them in subclasses, which is useful when you want to customize the extension's behavior for each subclass. The compiler handles each receiver type differently:

Receiver type	Resolution time	Dispatch type
Dispatch receiver	Runtime	Virtual
Extension receiver	Compile time	Static

Consider this example, where the `User` class is `open` and the `Admin` class inherits from it. The `NotificationSender` class defines `sendNotification()` extension functions for both `User` and `Admin` classes, and the `SpecialNotificationSender` class overrides them:

{kotlin-runnable="true" id="kotlin-extension-open"}

The dispatch receiver is resolved at runtime using virtual dispatch, which makes the behavior in the `main()` function easier to follow. What may surprise you is that when you call the `notify()` function on an `Admin` instance, the compiler chooses the extension based on the declared type: `user: User`, because it resolves the extension receiver statically.

Contoh Kode / Perintah CLI:

```
open class User

class Admin : User()

open class NotificationSender {
    open fun User.sendNotification() {
        println("Sending user notification from normal sender")
    }

    open fun Admin.sendNotification() {
        println("Sending admin notification from normal sender")
    }

    fun notify(user: User) {
        user.sendNotification()
    }
}

class SpecialNotificationSender : NotificationSender() {
    override fun User.sendNotification() {
        println("Sending user notification from special sender")
    }

    override fun Admin.sendNotification() {
        println("Sending admin notification from special sender")
    }
}

fun main() {
```

```
    // Dispatch receiver is NotificationSender
    // Extension receiver is User
    // Resolves to User.sendNotification() in NotificationSender
    NotificationSender().notify(User())
    // Sending user notification from normal sender

    // Dispatch receiver is SpecialNotificationSender
    // Extension receiver is User
    // Resolves to User.sendNotification() in SpecialNotificationSender
    SpecialNotificationSender().notify(User())
    // Sending user notification from special sender

    // Dispatch receiver is SpecialNotificationSender
    // Extension receiver is User NOT Admin
    // The notify() function declares user as type User
    // Statically resolves to User.sendNotification() in SpecialNotificationSender
    SpecialNotificationSender().notify(Admin())
    // Sending user notification from special sender
}
```

[kotlin_082] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Extensions and visibility modifiers

Sumber: ht

Extensions and visibility modifiers
Extensions use the same visibility modifiers as regular functions declared in the same scope, including extensions declared as members of other classes.
For example, an extension declared at the top level of a file can access other `private` top-level declarations in the same file:
{kotlin-runnable="true" id="kotlin-extension-visibility-top-level"}
And if an extension is declared outside its receiver type, it can't access the receiver's `private` or `protected` members:
{kotlin-runnable="true" id="kotlin-extension-visibility-outside-receiver"}
If an extension is marked as `internal`, it's only accessible within its module:

Contoh Kode / Perintah CLI:

```
// File: StringUtils.kt

private fun removeWhitespace(input: String): String {
    return input.replace("\\s".toRegex(), "")
}

fun String.cleaned(): String {
    return removeWhitespace(this)
}

fun main() {
    val rawEmail = "  user @example. com  "
    val cleaned = rawEmail.cleaned()
    println("Raw:     '$rawEmail'")
    // Raw:     '  user @example. com  '
    println("Cleaned: '$cleaned'")
    // Cleaned: 'user@example.com'
    println("Looks like an email: ${cleaned.contains("@") && cleaned.contains(".")}")
    // Looks like an email: true
}

// ----------------------------------------
```

```
class User(private val password: String) {
    fun isLoggedIn(): Boolean = true
    fun passwordLength(): Int = password.length
}

// Extension declared outside the class
fun User.isSecure(): Boolean {
    // Can't access password because it's private:
    // return password.length >= 8

    // Instead, we rely on public members:
    return passwordLength() >= 8 && isLoggedIn()
}

fun main() {
    val user = User("supersecret")
    println("Is user secure: ${user.isSecure()}")
    // Is user secure: true
}

// ----------------------------------------

// Networking module
// JsonParser.kt
internal fun String.parseJson(): Map<String, Any> {
    return mapOf("fakeKey" to "fakeValue")
}
```

[kotlin_083] Kotlin - Extension Functions & Properties

Extensions (Extension functions, Extension properties, Scope of extensions): Scope of extensions

Sumber: ht

Scope of extensions

In most cases, you define extensions on the top level, directly under packages:

To use an extension outside its declaring package, import it at the call site:

For more information, see Imports.

Contoh Kode / Perintah CLI:

```
package org.example.declarations

fun List<String>.getLongestString() { /*...*/}

// ----------------------------------------

package org.example.usage

import org.example.declarations.getLongestString

fun main() {
    val list = listOf("red", "green", "blue")
    list.getLongestString()
}
```

[kotlin_084] Kotlin - Asynchronous Programming - Coroutines

Coroutines (Lightweight threads, Suspending functions, Structured concurrency): Overview

Sumber: ht

[//]: # (title: Coroutines)
Applications often need to perform multiple tasks at the same time, such as responding to user input, loading data, or updating the screen.
To support this, they rely on concurrency, which allows operations to run independently without blocking each other.
The most common way to run tasks concurrently is by using threads, which are independent paths of execution managed by the operating system.
However, threads are relatively heavy, and creating many of them can lead to performance issues.
To support efficient concurrency, Kotlin uses asynchronous programming built around _coroutines_, which let you write asynchronous code in a natural, sequential style using suspending functions.
Coroutines are lightweight alternatives to threads.
They can suspend without blocking system resources and are resource-friendly, making them better suited for fine-grained concurrency.
Most coroutine features are provided by the `kotlinx.coroutines` library,
which includes tools for launching coroutines, handling concurrency, working with asynchronous streams, and more.
If you're new to coroutines in Kotlin, start with the Coroutine basics guide before diving into more complex topics.
This guide introduces the key concepts of suspending functions, coroutine builders, and structured concurrency through simple examples:
Get started with coroutines
> Check out the KotlinConf app for a sample project to see how coroutines are used in practice.
>
{style="tip"}

[kotlin_085] Kotlin - Asynchronous Programming - Coroutines

Coroutines (Lightweight threads, Suspending functions, Structured concurrency): Coroutine concepts

Sumber: ht

Coroutine concepts
The `kotlinx.coroutines` library provides the core building blocks for running tasks concurrently, structuring coroutine execution, and managing shared state.

[kotlin_086] Kotlin - Asynchronous Programming - Coroutines

Coroutines (Lightweight threads, Suspending functions, Structured concurrency): Suspending functions and coroutine builders

Sumber: ht

Suspending functions and coroutine builders
Coroutines in Kotlin are built on suspending functions, which allow code to pause and resume without blocking a thread.
The `suspend` keyword marks functions that can perform long-running operations asynchronously.
To launch new coroutines, use coroutine builders like `.launch()` and `.async()`.
These builders are extension functions on `CoroutineScope`,
which defines the coroutine's lifecycle and provides the coroutine context.
You can learn more about these builders in Coroutine basics and Composing suspend functions.

[kotlin_087] Kotlin - Asynchronous Programming - Coroutines

Coroutines (Lightweight threads, Suspending functions, Structured concurrency): Coroutine context and behavior

Sumber: ht

Coroutine context and behavior
Launching a coroutine from a `CoroutineScope` creates a context that governs its execution.
Builder functions like `.launch()` and `.async()` automatically create a set of elements that define how the coroutine behaves:
* The `Job` interface tracks the coroutine's lifecycle and enables structured concurrency.
* `CoroutineDispatcher` controls where the coroutine runs, such as on a background thread or the main thread in UI applications.
* `CoroutineExceptionHandler` handles uncaught exceptions.
These, along with other possible elements, make up the _coroutine context_, which is inherited by default from the coroutine's parent.
This context forms a hierarchy that enables structured concurrency, where related coroutines can be canceled together or handle exceptions as a group.

[kotlin_088] Kotlin - Asynchronous Programming - Coroutines

Coroutines (Lightweight threads, Suspending functions, Structured concurrency): Asynchronous flow and shared mutable state

Sumber: ht

Asynchronous flow and shared mutable state
Kotlin provides several ways for coroutines to communicate.
Use one of the following options based on how you want to share values between coroutines:
* `Flow` produces values only when a coroutine actively collects them.
* `Channel` allows multiple coroutines to send and receive values, with each value delivered to exactly one coroutine.
* `SharedFlow` continuously shares every value with all active collecting coroutines.
When multiple coroutines need to access or update the same data, they _share mutable state_.
Without coordination, this can lead to race conditions, where operations interfere with each other in unpredictable ways.
To safely manage shared mutable state, use `StateFlow` to wrap the shared data.
Then, you can update it from one coroutine and collect its latest value from others.
For more information, see Flows, Channels, and the Coroutines and channels tutorial.

[kotlin_089] Kotlin - Asynchronous Programming - Coroutines

Coroutines (Lightweight threads, Suspending functions, Structured concurrency): What's next

Sumber: ht

What's next
* Learn the fundamentals of coroutines, suspending functions, and builders in the Coroutine basics guide.
* Explore how to combine suspending functions and build coroutine pipelines in Composing suspending functions.
* Learn how to debug coroutines using built-in tools in IntelliJ IDEA.
* For flow-specific debugging, see the Debug Kotlin Flow using IntelliJ IDEA tutorial.
* Read the Guide to UI programming with coroutines to learn about coroutine-based UI development.
* Review best practices for using coroutines in Android.
* Check out the `kotlinx.coroutines` API reference.

[kotlin_090] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Overview

Sumber: ht

[//]: # (title: What's new in Kotlin 2.0.0)
Read the Kotlin 2.0.0 release notes covering new language features, updates to Kotlin Multiplatform, JVM, Native, JS, and Wasm, and build tool support for Gradle and Maven.
Released: May 21, 2024
The Kotlin 2.0.0 release is out and the new Kotlin K2 compiler is Stable! Additionally, here are
some other highlights:
* New Compose compiler Gradle plugin
* Generation of lambda functions using invokedynamic
* The kotlinx-metadata-jvm library is now Stable
* Monitoring GC performance in Kotlin/Native with signposts on Apple platforms
* Resolving conflicts in Kotlin/Native with Objective-C methods
* Support for named export in Kotlin/Wasm
* Support for unsigned primitive types in functions with @JsExport in Kotlin/Wasm
* Optimize production builds by default using Binaryen
* New Gradle DSL for compiler options in multiplatform projects
* Stable replacement of the enum class values generic function
* Stable AutoCloseable interface
Kotlin 2.0 is a huge milestone for the JetBrains team. This release was the center of KotlinConf 2024. Check out the opening
keynote, where we announced exciting updates and addressed the recent work on the Kotlin language:
> For information about the Kotlin release cycle, see Kotlin release process.
>
{style="tip"}

[kotlin_091] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): IDE support

Sumber: ht

IDE support
The Kotlin plugins that support Kotlin 2.0.0 are bundled in the latest IntelliJ IDEA and Android Studio.
You don't need to update the Kotlin plugin in your IDE.
All you need to do is to change the Kotlin version to Kotlin 2.0.0 in your build scripts.
* For details about IntelliJ IDEA's support for the Kotlin K2 compiler, see Support in IDEs.
* For more details about IntelliJ IDEA's support for Kotlin, see Kotlin releases.

[kotlin_092] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin K2 compiler

Sumber: ht

Kotlin K2 compiler
The road to the K2 compiler has been a long one, but now the JetBrains team is finally ready to announce its stabilization.
In Kotlin 2.0.0, the new Kotlin K2 compiler is used by default and it is Stable for all target
platforms: JVM, Native, Wasm, and JS. The new compiler brings major performance improvements, speeds up new language
feature development, unifies all platforms that Kotlin supports, and provides a better architecture for multiplatform

projects.
The JetBrains team has ensured the quality of the new compiler by successfully compiling 10 million lines of code from selected user and internal projects. 18,000 developers were involved in the stabilization process, testing the new K2 compiler across a total of 80,000 projects and reporting any problems they found.
To help make the migration process to the new compiler as smooth as possible, we've created a K2 compiler migration guide.
This guide explains the many benefits of the compiler, highlights any changes you might encounter, and describes how to
roll back to the previous version if necessary.
In a blog post,
we explored the performance of the K2 compiler in different projects. Check it out if you'd like to see real data on how the K2 compiler performs and find instructions on how to collect performance benchmarks from your own projects.
You can also watch this talk from KotlinConf 2024, where Michail Zare?enskij, the lead language designer, discusses the
feature evolution in Kotlin and the K2 compiler:

[kotlin_093] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Current K2 compiler limitations

Sumber: ht

Current K2 compiler limitations
Enabling K2 in your Gradle project comes with certain limitations that can affect projects using Gradle versions below 8.3 in the following cases:
* Compilation of source code from `buildSrc`.
* Compilation of Gradle plugins in included builds.
* Compilation of other Gradle plugins if they are used in projects with Gradle versions below 8.3.
* Building Gradle plugin dependencies.
If you encounter any of the problems mentioned above, you can take the following steps to address them:
* Set the language version for `buildSrc`, any Gradle plugins, and their dependencies:
> If you configure language and API versions for specific tasks, these values will override the values set by
> the `compilerOptions` extension. In this case, language and API versions should not be higher than 1.9.
>
{style="note"}
* Update the Gradle version in your project to 8.3 or later.

Contoh Kode / Perintah CLI:

```
kotlin {
      compilerOptions {
          languageVersion.set(org.jetbrains.kotlin.gradle.dsl.KotlinVersion.KOTLIN_1_9)
          apiVersion.set(org.jetbrains.kotlin.gradle.dsl.KotlinVersion.KOTLIN_1_9)
      }
  }
```

[kotlin_094] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Smart cast improvements

Sumber: ht

Smart cast improvements
The Kotlin compiler can automatically cast an object to a type in specific cases, saving you the trouble of having to explicitly cast it yourself. This is called smart casting.
The Kotlin K2 compiler now performs smart casts in even more scenarios than before.
In Kotlin 2.0.0, we've made improvements related to smart casts in the following areas:

* Local variables and further scopes
* Type checks with logical `or` operator
* Inline functions
* Properties with function types
* Exception handling
* Increment and decrement operators

Local variables and further scopes

Previously, if a variable was evaluated as not `null` within an `if` condition, the variable would be smart-cast. Information about this variable would then be shared further within the scope of the `if` block.

However, if you declared the variable **outside** the `if` condition, no information about the variable would be available within the `if` condition, so it couldn't be smart-cast. This behavior was also seen with `when` expressions and `while` loops.

From Kotlin 2.0.0, if you declare a variable before using it in your `if`, `when`, or `while` condition, then any information collected by the compiler about the variable will be accessible in the corresponding block for smart-casting.

This can be useful when you want to do things like extract boolean conditions into variables. Then, you can give the variable a meaningful name, which will improve your code readability and make it possible to reuse the variable later in your code. For example:

{kotlin-runnable="true" kotlin-min-compiler-version="2.0" id="kotlin-smart-casts-k2-local-variables" validate="false"}

Type checks with logical or operator

In Kotlin 2.0.0, if you combine type checks for objects with an `or` operat

Contoh Kode / Perintah CLI:

```
class Cat {
    fun purr() {
        println("Purr purr")
    }
}

fun petAnimal(animal: Any) {
    val isCat = animal is Cat
    if (isCat) {
        // In Kotlin 2.0.0, the compiler can access
        // information about isCat, so it knows that
        // animal was smart-cast to the type Cat.
        // Therefore, the purr() function can be called.
        // In Kotlin 1.9.20, the compiler doesn't know
        // about the smart cast, so calling the purr()
        // function triggers an error.
        animal.purr()
    }
}

fun main() {
    val kitty = Cat()
    petAnimal(kitty)
    // Purr purr
}

// ----------------------------------------

interface Status {
    fun signal() {}
}

interface Ok : Status
interface Postponed : Status
interface Declined : Status
```

```
fun signalCheck(signalStatus: Any) {
    if (signalStatus is Postponed || signalStatus is Declined) {
        // signalStatus is smart-cast to a common supertype Status
        signalStatus.signal()
        // Prior to Kotlin 2.0.0, signalStatus is smart cast
        // to type Any, so calling the signal() function triggered an
        // Unresolved reference error. The signal() function can only
        // be called successfully after another type check:

        // check(signalStatus is Status)
        // signalStatus.signal()
    }
}

// ----------------------------------------

interface Processor {
    fun process()
}

inline fun inlineAction(f: () -> Unit) = f()

fun nextProcessor(): Processor? = null

fun runProcessor(): Processor? {
    var processor: Processor? = null
    inlineAction {
        // In Kotlin 2.0.0, the compiler knows that processor
        // is a local variable, and inlineAction() is an inline function, so
        // references to processor can't be leaked. Therefore, it's safe
        // to smart-cast processor.

        // If processor isn't null, processor is smart-cast
        if (processor != null) {
            // The compiler knows that processor isn't null, so no safe call
            // is needed
            processor.process()

            // In Kotlin 1.9.20, you have to perform a safe call:
            // processor?.process()
        }

        processor = nextProcessor()
    }

    return processor
}
```

[kotlin_095] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin Multiplatform improvements

Sumber: ht

Kotlin Multiplatform improvements

In Kotlin 2.0.0, we've made improvements in the K2 compiler related to Kotlin Multiplatform in the following areas:

* Separation of common and platform sources during compilation
* Different visibility levels of expected and actual declarations

Separation of common and platform sources during compilation

Previously, the design of the Kotlin compiler prevented it from keeping common and platform source sets separate at compile time. As a consequence, common code could access platform code, which resulted in different behavior between platforms. In addition, some compiler settings and dependencies from common code used to leak into platform code.

In Kotlin 2.0.0, our implementation of the new Kotlin K2 compiler included a redesign of the compilation scheme to ensure

strict separation between common and platform source sets. This change is most noticeable when you use expected and actual functions.

Previously, it was possible for a function call in your common code to resolve to a function in platform code. For example:

Common code

Platform code

In this example, the common code has different behavior depending on which platform it is run on:

* On the JVM platform, calling the `foo()` function in the common code results in the `foo()` function from the platform code

being called as `platform foo`.

* On the JavaScript platform, calling the `foo()` function in the common code results in the `foo()` function from the common code being called as `common foo`, as there is no such function available in the platform code.

In Kotlin 2.0.0, common code doesn't have access to platform code, so both platforms successfully resolve the `foo()` function to the `foo()` function in the common code: `common foo`.

In addition to the improved consist

Contoh Kode / Perintah CLI:

```
fun foo(x: Any) = println("common foo")

fun exampleFunction() {
    foo(42)
}

// ----------------------------------------

// JVM
fun foo(x: Int) = println("platform foo")

// JavaScript
// There is no foo() function overload
// on the JavaScript platform

// ----------------------------------------

expect class Identity {
    fun confirmIdentity(): String
}

fun common() {
    // Before 2.0.0,
    // it triggers an IDE-only error
    Identity().confirmIdentity()
    // RESOLUTION_TO_CLASSIFIER : Expected class
    // Identity has no default constructor.
}
```

[kotlin_096] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Compiler plugins support

Sumber: ht

Compiler plugins support
Currently, the Kotlin K2 compiler supports the following Kotlin compiler plugins:
* `all-open`
* AtomicFU
* `jvm-abi-gen`
* `js-plain-objects`
* kapt
* Lombok
* `no-arg`
* Parcelize
* SAM with receiver
* serialization
* Power-assert
In addition, the Kotlin K2 compiler supports:
* The Jetpack Compose compiler plugin 2.0.0, which
was moved into the Kotlin repository.
* The Kotlin Symbol Processing (KSP) plugin since KSP2.
> If you use any additional compiler plugins, check their documentation to see if they are compatible with K2.
>
{style="tip"}

[kotlin_097] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Experimental Kotlin Power-assert compiler plugin

Sumber: ht

Experimental Kotlin Power-assert compiler plugin
> The Kotlin Power-assert plugin is Experimental.
> It may be changed at any time.
>
{style="warning"}
Kotlin 2.0.0 introduces an experimental Power-assert compiler plugin. This plugin improves the experience of writing tests by including contextual information in failure messages, making debugging easier and more efficient.
Developers often need to use complex assertion libraries to write effective tests. The Power-assert plugin simplifies this process by automatically generating failure messages that include intermediate values of the assertion expression. This helps developers quickly understand why a test failed.
When an assertion fails in a test, the improved error message shows the values of all variables and sub-expressions within the assertion, making it clear which part of the condition caused the failure. This is particularly useful for complex assertions where multiple conditions are checked.
To enable the plugin in your project, configure it in your `build.gradle(.kts)` file:
Learn more about the Kotlin Power-assert plugin in the documentation.

Contoh Kode / Perintah CLI:

```
plugins {
    kotlin("multiplatform") version "2.0.0"
    kotlin("plugin.power-assert") version "2.0.0"
}

powerAssert {
```

```
    functions = listOf("kotlin.assert", "kotlin.test.assertTrue")
}

// ----------------------------------------

plugins {
    id 'org.jetbrains.kotlin.multiplatform' version '2.0.0'
    id 'org.jetbrains.kotlin.plugin.power-assert' version '2.0.0'
}

powerAssert {
    functions = ["kotlin.assert", "kotlin.test.assertTrue"]
}
```

[kotlin_098] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): How to enable the Kotlin K2 compiler

Sumber: ht

How to enable the Kotlin K2 compiler
Starting with Kotlin 2.0.0, the Kotlin K2 compiler is enabled by default. No additional actions are required.

[kotlin_099] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Try the Kotlin K2 compiler in Kotlin Playground

Sumber: ht

Try the Kotlin K2 compiler in Kotlin Playground
Kotlin Playground supports the 2.0.0 release. Check it out!

[kotlin_100] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support in IDEs

Sumber: ht

Support in IDEs
By default, IntelliJ IDEA and Android Studio still use the previous compiler for code analysis, code completion, highlighting, and other IDE-related features. To get the full Kotlin 2.0 experience in your IDE, enable K2 mode.
In your IDE, go to **Settings** | **Languages & Frameworks** | **Kotlin** and select the **Enable K2 mode** option.
The IDE will analyze your code using its K2 mode.
!Enable K2 mode{width=200}
After enabling K2 mode, you may notice differences in IDE analysis due to changes in compiler behavior. Learn how the new K2 compiler differs from the previous one in our migration guide.
* Learn more about K2 mode in our blog.
* We are actively collecting feedback about K2 mode, so please share your thoughts in our public Slack channel.

[kotlin_101] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Leave your feedback on the new K2 compiler

Sumber: ht

Leave your feedback on the new K2 compiler
We would appreciate any feedback you may have!
* Report any problems you face with the new K2 compiler in our issue tracker.

* Enable the "Send usage statistics" option to
allow JetBrains to collect anonymous data about K2 usage.

[kotlin_102] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin/JVM

Sumber: ht

Kotlin/JVM
Starting with version 2.0.0, the compiler can generate classes containing Java 22 bytecode.
This version also brings the following changes:
* Generation of lambda functions using invokedynamic
* The kotlinx-metadata-jvm library is now Stable

[kotlin_103] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Generation of lambda functions using invokedynamic

Sumber: ht

Generation of lambda functions using invokedynamic
Kotlin 2.0.0 introduces a new default method for generating lambda functions using `invokedynamic`. This change reduces
the binary sizes of applications compared to the traditional anonymous class generation.
Since the first version, Kotlin has generated lambdas as anonymous classes. However, starting from Kotlin 1.5.0,
the option for `invokedynamic` generation has been available by using the `-Xlambdas=indy` compiler option. In Kotlin 2.0.0, `invokedynamic` has become the default method for lambda generation. This method produces lighter binaries and
aligns Kotlin with JVM optimizations, ensuring applications benefit from ongoing and future improvements in JVM performance.
Currently, it has three limitations compared to ordinary lambda compilation:
* A lambda compiled into `invokedynamic` is not serializable.
* Experimental `reflect()` API does not
support lambdas generated by `invokedynamic`.
* Calling `.toString()` on such a lambda produces a less readable string representation:
To retain the legacy behavior of generating lambda functions, you can either:
* Annotate specific lambdas with `@JvmSerializableLambda`.
* Use the compiler option `-Xlambdas=class` to generate all lambdas in a module using the legacy method.

Contoh Kode / Perintah CLI:

```
fun main() {
    println({})

    // With Kotlin 1.9.24 and reflection, returns
    // () -> kotlin.Unit

    // With Kotlin 2.0.0, returns
    // FileKt$$Lambda$13/0x00007f88a0004608@506e1b77
}
```

[kotlin_104] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): The kotlinx-metadata-jvm library is Stable

Sumber: ht

The kotlinx-metadata-jvm library is Stable
In Kotlin 2.0.0, the `kotlinx-metadata-jvm` library became Stable. Now that the library has changed to the `kotlin` package and coordinates, you can find it as `kotlin-metadata-jvm` (without the "x").
Previously, the `kotlinx-metadata-jvm` library had its own publishing scheme and version. Now, we will build and publish the `kotlin-metadata-jvm` updates as part of the Kotlin release cycle, with the same backward compatibility guarantees as the Kotlin standard library.
The `kotlin-metadata-jvm` library provides an API to read and modify metadata of binary files generated by the Kotlin/JVM compiler.

[kotlin_105] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin/Native

Sumber: ht

Kotlin/Native
This version brings the following changes:
* Monitoring GC performance with signposts
* Resolving conflicts with Objective-C methods
* Changed log level for compiler arguments in Kotlin/Native
* Explicitly added standard library and platform dependencies to Kotlin/Native
* Tasks error in Gradle configuration cache

[kotlin_106] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Monitoring GC performance with signposts on Apple platforms

Sumber: ht

Monitoring GC performance with signposts on Apple platforms
Previously, it was only possible to monitor the performance of Kotlin/Native's garbage collector (GC) by looking into logs. However, these logs were not integrated with Xcode Instruments, a popular toolkit for investigating issues with iOS apps' performance.
Since Kotlin 2.0.0, GC reports pauses with signposts that are available in Instruments. Signposts allow for custom logging within your app, so now, when debugging iOS app performance, you can check if a GC pause corresponds to the
application freeze.
Learn more about GC performance analysis in the documentation.

[kotlin_107] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Resolving conflicts with Objective-C methods

Sumber: ht

Resolving conflicts with Objective-C methods
Objective-C methods can have different names, but the same number and types of parameters. For example, `locationManager:didEnterRegion:`
and `locationManager:didExitRegion:`.
In Kotlin, these methods have the same signature, so an attempt to use them triggers a conflicting overloads error. Previously, you had to manually suppress conflicting overloads to avoid this compilation error. To improve Kotlin

interoperability with Objective-C, the Kotlin 2.0.0 introduces the new `@ObjCSignatureOverride` annotation.
The annotation instructs the Kotlin compiler to ignore conflicting overloads, in case several functions with the same argument types but different argument names are inherited from the Objective-C class.
Applying this annotation is also safer than general error suppression. This annotation can only be used in the case of overriding Objective-C methods, which are supported and tested, while general suppression may hide important errors and
lead to silently broken code.

[kotlin_108] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Changed log level for compiler arguments

Sumber: ht

Changed log level for compiler arguments
In this release, the log level for compiler arguments in Kotlin/Native Gradle tasks, such as `compile`, `link`, and `cinterop`, has changed from `info` to `debug`.
With `debug` as its default value, the log level is consistent with other Gradle compilation tasks and provides detailed debugging information, including all compiler arguments.

[kotlin_109] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Explicitly added standard library and platform dependencies to Kotlin/Native

Sumber: ht

Explicitly added standard library and platform dependencies to Kotlin/Native
Previously, the Kotlin/Native compiler resolved standard library and platform dependencies implicitly, which caused inconsistencies in the way the Kotlin Gradle plugin worked across Kotlin targets.
Now, each Kotlin/Native Gradle compilation explicitly includes standard library and platform dependencies in its compile-time library path via the `compileDependencyFiles` compilation parameter.

[kotlin_110] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Tasks error in Gradle configuration cache

Sumber: ht

Tasks error in Gradle configuration cache
Since Kotlin 2.0.0, you may encounter a configuration cache error with messages indicating:
`invocation of Task.project at execution time is unsupported`.
This error appears in tasks such as `NativeDistributionCommonizerTask` and `KotlinNativeCompile`.
However, this is a false-positive error. The underlying issue is the presence of tasks that are not compatible with the Gradle configuration cache, like the `publish*` task.
This discrepancy may not be immediately apparent, as the error message suggests a different root cause.
As the precise cause isn't explicitly stated in the error report, the Gradle team is already addressing the issue to fix reports.

[kotlin_111] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin/Wasm

Sumber: ht

Kotlin/Wasm
Kotlin 2.0.0 improves performance and interoperability with JavaScript:
* Optimized production builds by default using Binaryen
* Support for named export
* Support for unsigned primitive types in functions with `@JsExport`
* Generation of TypeScript declaration files in Kotlin/Wasm
* Support for catching JavaScript exceptions
* New exception handling proposal is now supported as an option
* The `withWasm()` function is split into JS and WASI variants

[kotlin_112] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Optimized production builds by default using Binaryen

Sumber: ht

Optimized production builds by default using Binaryen
The Kotlin/Wasm toolchain now applies the Binaryen tool during production
compilation to all projects, as opposed to the previous manual setup approach. By our estimations, it should improve runtime performance and reduce the binary size for your project.
> This change only affects production compilation. The development compilation process stays the same.
>
{style="note"}

[kotlin_113] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support for named export

Sumber: ht

Support for named export
Previously, all exported declarations from Kotlin/Wasm were imported into JavaScript using default export:
Now, you can import each Kotlin declaration marked with `@JsExport` by name:
Named exports make it easier to share code between Kotlin and JavaScript modules. They improve readability and help you
manage dependencies between modules.

Contoh Kode / Perintah CLI:
```
Now, you can import each Kotlin declaration marked with `@JsExport` by name:
```

[kotlin_114] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support for unsigned primitive types in functions with @JsExport

Sumber: ht

Support for unsigned primitive types in functions with @JsExport
Starting from Kotlin 2.0.0, you can use unsigned primitive types inside external
declarations and functions with the `@JsExport` annotation that makes Kotlin/Wasm functions available in JavaScript code.
This helps to mitigate the previous limitation that prevented the unsigned primitives from

being used directly inside exported and external declarations. Now you can export functions with unsigned primitives as a return or parameter type and consume external declarations that return or consume unsigned primitives.
For more information on Kotlin/Wasm interoperability with JavaScript, see the documentation.

[kotlin_115] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Generation of TypeScript declaration files in Kotlin/Wasm

Sumber: ht

Generation of TypeScript declaration files in Kotlin/Wasm
> Generating TypeScript declaration files in Kotlin/Wasm is Experimental.
> It may be dropped or changed at any time.
>
{style="warning"}
In Kotlin 2.0.0, the Kotlin/Wasm compiler is now capable of generating TypeScript definitions from any `@JsExport` declarations in your Kotlin code. These definitions can be used by IDEs and JavaScript tools to provide code autocompletion, help with type checks, and make it easier to include Kotlin code in JavaScript.
The Kotlin/Wasm compiler collects any top-level functions
marked with `@JsExport` and automatically generates TypeScript definitions in a `.d.ts` file.
To generate TypeScript definitions, in your `build.gradle(.kts)` file in the `wasmJs {}` block, add
the `generateTypeScriptDefinitions()` function:

Contoh Kode / Perintah CLI:

```
kotlin {
    wasmJs {
        binaries.executable()
        browser {
        }
        generateTypeScriptDefinitions()
    }
}
```

[kotlin_116] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support for catching JavaScript exceptions

Sumber: ht

Support for catching JavaScript exceptions
Previously, Kotlin/Wasm code could not catch JavaScript exceptions, making it difficult to handle errors originating from the JavaScript side of the program.
In Kotlin 2.0.0, we have implemented support for catching JavaScript exceptions within Kotlin/Wasm. This implementation
allows you to use `try-catch` blocks, with specific types like `Throwable` or `JsException`, to handle these errors properly.
Additionally, `finally` blocks, which help execute code regardless of exceptions, also work correctly. While we're introducing support for catching JavaScript exceptions, no additional information is provided when a JavaScript exception,
like a call stack, occurs. However, we are working on these implementations.

[kotlin_117] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New exception handling proposal is now supported as an option

Sumber: ht

New exception handling proposal is now supported as an option
In this release, we introduce support for the new version of WebAssembly's exception handling proposal
within Kotlin/Wasm.
This update ensures the new proposal aligns with Kotlin requirements, enabling the use of Kotlin/Wasm on virtual machines that only support the latest version of the proposal.
Activate the new exception handling proposal by using the `-Xwasm-use-new-exception-proposal` compiler option, which is
turned off by default.

[kotlin_118] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): The withWasm() function is split into JS and WASI variants

Sumber: ht

The withWasm() function is split into JS and WASI variants
The `withWasm()` function, which used to provide Wasm targets for hierarchy templates, is deprecated in favor of specialized `withWasmJs()` and `withWasmWasi()` functions.
Now you can separate the WASI and JS targets between different groups in the tree definition.

[kotlin_119] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin/JS

Sumber: ht

Kotlin/JS
Among other changes, this version brings modern JS compilation to Kotlin, supporting more features from the ES2015 standard:
* New compilation target
* Suspend functions as ES2015 generators
* Passing arguments to the main function
* Per-file compilation for Kotlin/JS projects
* Improved collection interoperability
* Support for createInstance()
* Support for type-safe plain JavaScript objects
* Support for npm package manager
* Changes to compilation tasks
* Discontinuing legacy Kotlin/JS JAR artifacts

[kotlin_120] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New compilation target

Sumber: ht

New compilation target
In Kotlin 2.0.0, we're adding a new compilation target to Kotlin/JS, `es2015`. This is a new way for you to enable all the ES2015 features supported in Kotlin at once.
You can set it up in your `build.gradle(.kts)` file like this:
The new target automatically turns on ES classes and modules

and the newly supported ES generators.

Contoh Kode / Perintah CLI:

```
kotlin {
    js {
        compilerOptions {
            target.set("es2015")
        }
    }
}
```

[kotlin_121] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Suspend functions as ES2015 generators

Sumber: ht

Suspend functions as ES2015 generators
This release introduces Experimental support for ES2015 generators
for compiling suspend functions.
Using generators instead of state machines should improve the final bundle size of your project. For example, the JetBrains team managed to decrease the bundle size of its Space project by 20% by using the ES2015 generators.
Learn more about ES2015 (ECMAScript 2015, ES6) in the official documentation.

[kotlin_122] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Passing arguments to the main function

Sumber: ht

Passing arguments to the main function
Starting with Kotlin 2.0.0, you can specify a source of your `args` for the `main()` function. This feature makes it easier to work with the command line and pass the arguments.
To do this, define the `js {}` block with the new `passAsArgumentToMainFunction()` function, which returns an array of strings:
The function is executed at runtime. It takes the JavaScript expression and uses it as the `args: Array` argument instead of the `main()` function call.
Also, if you use the Node.js runtime, you can take advantage of a special alias. It allows you to pass `process.argv` to the `args` parameter once instead of adding it manually every time:

Contoh Kode / Perintah CLI:

```
kotlin {
    js {
        binary.executable()
        passAsArgumentToMainFunction("Deno.args")
    }
}

// ----------------------------------------

kotlin {
    js {
        binary.executable()
        nodejs {
            passProcessArgvToMainFunction()
        }
    }
}
```

[kotlin_123] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Per-file compilation for Kotlin/JS projects

Sumber: ht

Per-file compilation for Kotlin/JS projects
Kotlin 2.0.0 introduces a new granularity option for the Kotlin/JS project output. You can now set up a per-file compilation that generates one JavaScript file for each Kotlin file. It helps to significantly optimize the size of the final bundle and improve the loading time of the program.
Previously, there were only two output options. The Kotlin/JS compiler could generate a single `.js` file for the whole project. However, this file might be too large and inconvenient to use. Whenever you wanted to use a function from your project, you had to include the entire JavaScript file as a dependency. Alternatively, you could configure a compilation of a separate `.js` file for each project module. This is still the default option.
Since module files could also be too large, with Kotlin 2.0.0, we add a more granular output that generates one (or two, if the file contains exported declarations) JavaScript file per each Kotlin file. To enable the per-file compilation mode:
1. Add the `useEsModules()` function to your build
file to support ECMAScript modules:
You can also use the new `es2015` compilation target for that.
2. Apply the `-Xir-per-file` compiler option or update your `gradle.properties` file with:

Contoh Kode / Perintah CLI:

```
// build.gradle.kts
   kotlin {
       js(IR) {
           useEsModules() // Enables ES2015 modules
           browser()
       }
   }
```

[kotlin_124] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Improved collection interoperability

Sumber: ht

Improved collection interoperability
Starting with Kotlin 2.0.0, it's possible to export declarations with a Kotlin collection type inside the signature to JavaScript (and TypeScript). This applies to `Set`, `Map`, and `List` collection types and their mutable counterparts.
To use Kotlin collections in JavaScript, first mark the necessary declarations
with `@JsExport` annotation:
You can then consume them from JavaScript as regular JavaScript arrays:
> Unfortunately, creating Kotlin collections from JavaScript is still unavailable. We're planning to add this
> functionality in Kotlin 2.0.20.
>
{style="note"}

Contoh Kode / Perintah CLI:

```
// Kotlin
@JsExport
data class User(
    val name: String,
    val friends: List<User> = emptyList()
)

@JsExport
val me = User(
    name = "Me",
    friends = listOf(User(name = "Kodee"))
```

```
)
```

[kotlin_125] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support for createInstance()

Sumber: ht

Support for createInstance()
Starting with Kotlin 2.0.0, you can use the `createInstance()`
function from the Kotlin/JS target. Previously, it was only available on the JVM.
This function from the KClass interface creates
a new instance of the specified class, which is useful for getting the runtime reference to a Kotlin class.

[kotlin_126] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support for type-safe plain JavaScript objects

Sumber: ht

Support for type-safe plain JavaScript objects
> The `js-plain-objects` plugin is Experimental.
> It may be dropped or changed at any time. The `js-plain-objects` plugin **only** supports the K2 compiler.
>
{style="warning"}
To make it easier to work with JavaScript APIs, in Kotlin 2.0.0, we provide a new plugin: `js-plain-objects`,
which you can use to create type-safe plain JavaScript objects. The plugin checks your code for any external interfaces
that have a `@JsPlainObject` annotation and adds:
* An inline `invoke` operator function inside the companion object that you can use as a constructor.
* A `.copy()` function that you can use to create a copy of your object while adjusting some of its properties.
For example:
Any JavaScript objects created with this approach are safer because instead of only seeing errors at runtime,
you can see them at compile time or even highlighted by your IDE.
Consider this example, which uses a `fetch()` function to interact with a JavaScript API using external interfaces
to describe the shape of the JavaScript objects:
In comparison, if you use the `js()` function instead to create your JavaScript objects,
errors are only found at runtime or aren't triggered at all:
To use the `js-plain-objects` plugin, add the following to your `build.gradle(.kts)` file:

Contoh Kode / Perintah CLI:

```
import kotlinx.js.JsPlainObject

@JsPlainObject
external interface User {
    var name: String
    val age: Int
    val email: String?
}

fun main() {
    // Creates a JavaScript object
    val user = User(name = "Name", age = 10)
    // Copies the object and adds an email
    val copy = user.copy(age = 11, email = "some@user.com")

    println(JSON.stringify(user))
    // { "name": "Name", "age": 10 }
    println(JSON.stringify(copy))
```

```
    // { "name": "Name", "age": 11, "email": "some@user.com" }
}

// ----------------------------------------

import kotlinx.js.JsPlainObject

@JsPlainObject
external interface FetchOptions {
    val body: String?
    val method: String
}

// A wrapper for Window.fetch
suspend fun fetch(url: String, options: FetchOptions? = null) = TODO("Add your custom behavior here")

// A compile-time error is triggered as "metod" is not recognized
// as method
fetch("https://google.com", options = FetchOptions(metod = "POST"))
// A compile-time error is triggered as method is required
fetch("https://google.com", options = FetchOptions(body = "SOME STRING"))

// ----------------------------------------

suspend fun fetch(url: String, options: FetchOptions? = null) = TODO("Add your custom behavior here")

// No error is triggered. As "metod" is not recognized, the wrong method
// (GET) is used.
fetch("https://google.com", options = js("{ metod: 'POST' }"))

// By default, the GET method is used. A runtime error is triggered as
// body shouldn't be present.
fetch("https://google.com", options = js("{ body: 'SOME STRING' }"))
// TypeError: Window.fetch: HEAD or GET Request cannot have a body
```

[kotlin_127] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Support for npm package manager

Sumber: ht

Support for npm package manager

Previously, it was only possible for the Kotlin Multiplatform Gradle plugin to use Yarn
as a package manager to download and install npm dependencies. From Kotlin 2.0.0, you can use npm
as your package manager instead. Using npm as a package manager means that you have one less tool to manage during your
setup.

For backward compatibility, Yarn is still the default package manager. To use npm as your package manager,
set the following property in your `gradle.properties` file:

Contoh Kode / Perintah CLI:

```
kotlin.js.yarn = false
```

[kotlin_128] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Changes to compilation tasks

Sumber: ht

Changes to compilation tasks
Previously, the `webpack` and `distributeResources` compilation tasks both targeted the same directories. Moreover, the `distribution` task declared the `dist` as its output directory as well. This resulted in overlapping outputs and produced a compilation warning.
So, starting with Kotlin 2.0.0, we've implemented the following changes:
* The `webpack` task now targets a separate folder.
* The `distributeResources` task has been completely removed.
* The `distribution` task now has the `Copy` type and targets the `dist` folder.

[kotlin_129] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Discontinuing legacy Kotlin/JS JAR artifacts

Sumber: ht

Discontinuing legacy Kotlin/JS JAR artifacts
Starting with Kotlin 2.0.0, the Kotlin distribution no longer contains legacy Kotlin/JS artifacts with the `.jar`
extension. Legacy artifacts were used in the unsupported old Kotlin/JS compiler and unnecessary for the IR compiler, which
uses the `klib` format.

[kotlin_130] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Gradle improvements

Sumber: ht

Gradle improvements
Kotlin 2.0.0 is fully compatible with Gradle 6.8.3 through 8.5. You can also use Gradle versions up to the latest Gradle release, but if you do, keep in mind that you might encounter deprecation warnings or some new Gradle features might not
work.
This version brings the following changes:
* New Gradle DSL for compiler options in multiplatform projects
* New Compose compiler Gradle plugin
* New attribute to distinguish JVM and Android published libraries
* Improved Gradle dependency handling for CInteropProcess in Kotlin/Native
* Visibility changes in Gradle
* New directory for Kotlin data in Gradle projects
* Kotlin/Native compiler downloaded when needed
* Deprecating old ways of defining compiler options
* Bumped minimum AGP supported version
* New Gradle property for trying the latest language version
* New JSON output format for build reports
* kapt configurations inherit annotation processors from superconfigurations
* Kotlin Gradle plugin no longer uses deprecated Gradle conventions

[kotlin_131] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New Gradle DSL for compiler options in multiplatform projects

Sumber: ht

New Gradle DSL for compiler options in multiplatform projects
> This feature is Experimental. It may be dropped or changed at
> any time. Use it only for evaluation purposes. We would appreciate your feedback on it in YouTrack.
>
{style="warning"}
Prior to Kotlin 2.0.0, configuring compiler options in a multiplatform project with Gradle was only possible at a low level, such as per task, compilation, or source set. To make it easier to configure compiler options more generally in your projects, Kotlin 2.0.0 comes with a new Gradle DSL.
With this new DSL, you can configure compiler options at the extension level for all the targets and shared source sets like `commonMain` and at a target level for a specific target:
The overall project configuration now has three layers. The highest is the extension level, then the target level and the lowest is the compilation unit (which is usually a compilation task):
!Kotlin compiler options levels{width=700}
The settings at a higher level are used as a convention (default) for a lower level:
* The values of extension compiler options are the default for target compiler options, including shared source sets, like `commonMain`, `nativeMain`, and `commonTest`.
* The values of target compiler options are used as the default for compilation unit (task) compiler options, for example, `compileKotlinJvm` and `compileTestKotlinJvm` tasks.
In turn, configurations made at a lower level override related settings at a higher level:
* Task-level compiler options override related configurations at the target or the extension level.
* Target-level compiler options override related configurations at the extension level.
When configuring your project, keep in mind that some old ways of setting up compiler options have been deprecated. We encourag

Contoh Kode / Perintah CLI:

```
kotlin {
    compilerOptions {
        // Extension-level common compiler options that are used as defaults
        // for all targets and shared source sets
        allWarningsAsErrors.set(true)
    }
    jvm {
        compilerOptions {
            // Target-level JVM compiler options that are used as defaults
            // for all compilations in this target
            noJdk.set(true)
        }
    }
}
```

[kotlin_132] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New Compose compiler Gradle plugin

Sumber: ht

New Compose compiler Gradle plugin
The Jetpack Compose compiler, which translates composables into Kotlin code, has now been merged into the Kotlin repository. This will help transition Compose projects to Kotlin 2.0.0, as the Compose compiler will always ship simultaneously with Kotlin. This also bumps the Compose compiler version to 2.0.0.
To use the new Compose compiler in your projects, apply the `org.jetbrains.kotlin.plugin.compose` Gradle plugin in

your `build.gradle(.kts)` file and set its version equal to Kotlin 2.0.0.
To learn more about this change and see the migration instructions, see the Compose compiler documentation.

[kotlin_133] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New attribute to distinguish JVM and Android-published libraries

Sumber: ht

New attribute to distinguish JVM and Android-published libraries
Starting with Kotlin 2.0.0, the `org.gradle.jvm.environment`
Gradle attribute is published by default with all Kotlin variants.
The attribute helps distinguish JVM and Android variants of Kotlin Multiplatform libraries. It indicates that a certain library variant is better suited for a certain JVM environment. The target environment could be "android", "standard-jvm", or "no-jvm".
Publishing this attribute should make consuming Kotlin Multiplatform libraries with JVM and Android targets more robust from non-multiplatform clients as well, such as Java-only projects.
If necessary, you can disable attribute publication. To do that, add the following Gradle option to
your `gradle.properties` file:

[kotlin_134] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Improved Gradle dependency handling for CInteropProcess in Kotlin/Native

Sumber: ht

Improved Gradle dependency handling for CInteropProcess in Kotlin/Native
In this release, we enhanced the handling of the `defFile` property to ensure better Gradle task dependency management
in Kotlin/Native projects.
Before this update, Gradle builds could fail if the `defFile` property was designated as an output
of another task that hadn't been executed yet. The workaround for this issue was to add a dependency on this task:
To fix this, there is a new `RegularFileProperty` property called `definitionFile`. Now, Gradle lazily verifies the presence of the `definitionFile` property after the connected task has run later in the build process. This new approach eliminates the need for additional dependencies.
The `CInteropProcess` task and the `CInteropSettings` class use the `definitionFile` property instead of `defFile` and `defFileProperty`:
> `defFile` and `defFileProperty` parameters are deprecated.
>
{style="warning"}

Contoh Kode / Perintah CLI:

```
kotlin {
    macosArm64("native") {
        compilations.getByName("main") {
            cinterops {
                val cinterop by creating {
                    defFileProperty.set(createDefFileTask.flatMap { it.defFile.asFile })
                    project.tasks.named(interopProcessingTaskName).configure {
                        dependsOn(createDefFileTask)
                    }
                }
            }
        }
    }
}
```

```
// ----------------------------------------

kotlin {
    macosArm64("native") {
        compilations.getByName("main") {
            cinterops {
                val cinterop by creating {
                    definitionFile.set(project.file("def-file.def"))
                }
            }
        }
    }
}

// ----------------------------------------

kotlin {
    macosArm64("native") {
        compilations.main {
            cinterops {
                cinterop {
                    definitionFile.set(project.file("def-file.def"))
                }
            }
        }
    }
}
```

[kotlin_135] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Visibility changes in Gradle

Sumber: ht

Visibility changes in Gradle
> This change impacts only Kotlin DSL users.
>
{style="note"}
In Kotlin 2.0.0, we've modified the Kotlin Gradle Plugin for better control and safety in your build scripts. Previously, certain Kotlin DSL functions and properties intended for a specific DSL context would inadvertently leak into other DSL contexts. This leakage could lead to the use of incorrect compiler options, settings being applied multiple times, and other misconfigurations:
To fix this issue, we've added the `@KotlinGradlePluginDsl` annotation, preventing the exposure of the Kotlin Gradle plugin DSL functions and properties to levels where they are not intended to be available. The following levels are separated from each other:
* Kotlin extension
* Kotlin target
* Kotlin compilation
* Kotlin compilation task
For the most popular cases, we've added compiler warnings with suggestions on how to fix them if your build script is configured incorrectly. For example:
In this case, the warning message for `sourceSets` is:
We would appreciate your feedback on this change! Share your comments directly to Kotlin developers in our #gradle Slack channel. Get a Slack invite.

Contoh Kode / Perintah CLI:

```
kotlin {
```

```
    // Target DSL couldn't access methods and properties defined in the
    // kotlin{} extension DSL
    jvm {
        // Compilation DSL couldn't access methods and properties defined
        // in the kotlin{} extension DSL and Kotlin jvm{} target DSL
        compilations.configureEach {
            // Compilation task DSLs couldn't access methods and
            // properties defined in the kotlin{} extension, Kotlin jvm{}
            // target or Kotlin compilation DSL
            compileTaskProvider.configure {
                // For example:
                explicitApi()
                // ERROR as it is defined in the kotlin{} extension DSL
                mavenPublication {}
                // ERROR as it is defined in the Kotlin jvm{} target DSL
                defaultSourceSet {}
                // ERROR as it is defined in the Kotlin compilation DSL
            }
        }
    }
}

// ---------------------------------------

kotlin {
    jvm {
        sourceSets.getByName("jvmMain").dependencies {
            implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core-jvm:1.7.3")
        }
    }
}
```

[kotlin_136] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New directory for Kotlin data in Gradle projects

Sumber: ht

New directory for Kotlin data in Gradle projects
> Do not commit the `.kotlin` directory to version control.
> For example, if you are using Git, add `.kotlin` to your project's `.gitignore` file.
>
{style="warning"}
In Kotlin 1.8.20, the Kotlin Gradle plugin switched to storing its data in the Gradle project cache directory: `/.gradle/kotlin`. However, the `.gradle` directory is reserved for Gradle only, and as a result it's not future-proof.
To solve this, as of Kotlin 2.0.0, we will store Kotlin data in your `/.kotlin` by default.
We will continue to store some data in the `.gradle/kotlin` directory for backward compatibility.
The new Gradle properties you can configure are:

Gradle property	Description
`kotlin.project.persistent.dir`	Configures the location where your project-level data is stored. Default: `/.kotlin`
`kotlin.project.persistent.dir.gradle.disableWrite`	A boolean value that controls whether writing Kotlin data to the `.gradle` directory is disabled. Default: `false`

Add these properties to the `gradle.properties` file in your projects for them to take effect.

[kotlin_137] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin/Native compiler downloaded when needed

Sumber: ht

Kotlin/Native compiler downloaded when needed
Before Kotlin 2.0.0, if you had a Kotlin/Native target configured in the Gradle build script of your multiplatform project, Gradle would always download the Kotlin/Native compiler in the configuration phase.
This happened even if there was no task to compile code for a Kotlin/Native target that was due to run in the execution phase.
Downloading the Kotlin/Native compiler in this way was particularly inefficient for users who only wanted to check the JVM or JavaScript code in their projects. For example, to perform tests or checks with their Kotlin project as part of a CI process.
In Kotlin 2.0.0, we changed this behavior in the Kotlin Gradle plugin so that the Kotlin/Native compiler is downloaded in the execution phase
and **only** when a compilation is requested for a Kotlin/Native target.
In turn, the Kotlin/Native compiler's dependencies are now downloaded not as a part of the compiler, but in the execution phase as well.
If you encounter any issues with the new behavior, you can temporarily switch back to the previous behavior by adding the following Gradle property to your `gradle.properties` file:
Starting with Kotlin 1.9.20-Beta, the Kotlin/Native distribution is published to Maven Central
along with the CDN.
This allowed us to change how Kotlin looks for and downloads the necessary artifacts. Instead of the CDN, by default, it now uses the Maven repositories that you specified in the `repositories {}` block of your project.
You can temporarily switch this behavior back by setting the following Gradle property in your `gradle.properties` file:
Please report any problems to our issue tracker YouTrack. Both of these Gradle properties that
change the default behavior are temporary and will be removed in future releas

Contoh Kode / Perintah CLI:

```
Starting with Kotlin 1.9.20-Beta, the Kotlin/Native distribution is published to [Maven Central](https://repo.maven.apache.org/maven2/org/jetbrains/kotlin/kotlin-native-prebuilt/)
along with the CDN.

This allowed us to change how Kotlin looks for and downloads the necessary artifacts. Instead of the CDN, by default,
it now uses the Maven repositories that you specified in the `repositories {}` block of your project.

You can temporarily switch this behavior back by setting the following Gradle property in your `gradle.properties` file:
```

[kotlin_138] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Deprecated old ways of defining compiler options

Sumber: ht

Deprecated old ways of defining compiler options
In this release, we continue to refine how you can set up compiler options. It should resolve ambiguity between different ways and make the project configuration more straightforward.
Since Kotlin 2.0.0, the following DSLs for specifying compiler options are deprecated:
* The `kotlinOptions` DSL from the `KotlinCompile` interface that implements all Kotlin compilation tasks. Use `KotlinCompilationTask` instead.
* The `compilerOptions` property with the `HasCompilerOptions` type from the `KotlinCompilation` interface. This DSL was
inconsistent with other DSLs and configured the same `KotlinCommonCompilerOptions` object as `compilerOptions` inside

the `KotlinCompilation.compileTaskProvider` compilation task, which was confusing.
Instead, we recommend using the `compilerOptions` property from the
Kotlin compilation task:
For example:
* The `kotlinOptions` DSL from the `KotlinCompilation` interface.
* The `kotlinOptions` DSL from the `KotlinNativeArtifactConfig` interface, the `KotlinNativeLink` class,
and the `KotlinNativeLinkArtifactTask` class. Use the `toolOptions` DSL instead.
* The `dceOptions` DSL from the `KotlinJsDce` interface. Use the `toolOptions` DSL instead.
For more information on how to specify compiler options in the Kotlin Gradle plugin, see How to define options.

Contoh Kode / Perintah CLI:

```
kotlinCompilation.compileTaskProvider.configure {
      compilerOptions { ... }
  }

// ----------------------------------------

kotlin {
      js(IR) {
          compilations.all {
              compileTaskProvider.configure {
                  compilerOptions.freeCompilerArgs.add("-Xir-minimized-member-names=false")
              }
          }
      }
  }
```

[kotlin_139] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Bumped minimum supported AGP version

Sumber: ht

Bumped minimum supported AGP version
Starting with Kotlin 2.0.0, the minimum supported Android Gradle plugin version is 7.1.3.

[kotlin_140] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New Gradle property for trying the latest language version

Sumber: ht

New Gradle property for trying the latest language version
Prior to Kotlin 2.0.0, we had the following Gradle property to try out the new K2 compiler: `kotlin.experimental.tryK2`.
Now that the K2 compiler is enabled by default in Kotlin 2.0.0, we decided to evolve this property into a new form that you can use to try the latest language version in your projects: `kotlin.experimental.tryNext`. When you use this property in your `gradle.properties` file, the Kotlin Gradle plugin increments the language version to one above the default value for your Kotlin version. For example, in Kotlin 2.0.0, the default language version is 2.0, so the property configures language version 2.1.
This new Gradle property produces similar metrics in build reports as
before with `kotlin.experimental.tryK2`. The language version configured is included in the output. For example:
To learn more about how to enable build reports and their content, see Build reports.

[kotlin_141] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): New JSON output format for build reports

Sumber: ht

New JSON output format for build reports
In Kotlin 1.7.0, we introduced build reports to help track compiler performance. Over time, we've added more metrics to make these reports even more detailed and helpful when investigating performance issues. Previously, the only output format for a local file was the `*.txt` format. In Kotlin 2.0.0, we support the JSON output format to make it even easier to analyze using other tools.
To configure JSON output format for your build reports, declare the following properties in your `gradle.properties` file:
Alternatively, you can run the following command:
Once configured, Gradle generates your build reports in the directory that you specify with the name: `${project_name}-date-time-.json`.
Here's an example snippet from a build report with JSON output format that contains build metrics and aggregated metrics:

Contoh Kode / Perintah CLI:

```
Alternatively, you can run the following command:

// ----------------------------------------

Once configured, Gradle generates your build reports in the directory that you specify with the name:
`${project_name}-date-time-<sequence_number>.json`.

Here's an example snippet from a build report with JSON output format that contains build metrics and
aggregated
metrics:
```

[kotlin_142] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): kapt configurations inherit annotation processors from superconfigurations

Sumber: ht

kapt configurations inherit annotation processors from superconfigurations
Prior to Kotlin 2.0.0, if you wanted to define a common set of annotation processors in a separate Gradle configuration and extend this configuration in kapt-specific configurations for your subprojects, kapt would skip annotation processing because it couldn't find any annotation processors. In Kotlin 2.0.0, kapt can successfully detect that there are indirect dependencies on your annotation processors.
As an example, for a subproject using Dagger, in your `build.gradle(.kts)` file, use the following configuration:
In this example, the `commonAnnotationProcessors` Gradle configuration is your common configuration for annotation processing that you want to be used for all your projects. You use the `extendsFrom()` method to add `commonAnnotationProcessors` as a superconfiguration. kapt sees that the `commonAnnotationProcessors` Gradle configuration has a dependency on the Dagger annotation processor. Therefore, kapt includes the Dagger annotation processor in its configuration for annotation processing.
Thanks to Christoph Loy for the implementation!

Contoh Kode / Perintah CLI:

```
val commonAnnotationProcessors by configurations.creating
configurations.named("kapt") { extendsFrom(commonAnnotationProcessors) }

dependencies {
```

```
    implementation("com.google.dagger:dagger:2.48.1")
    commonAnnotationProcessors("com.google.dagger:dagger-compiler:2.48.1")
}
```

[kotlin_143] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Kotlin Gradle plugin no longer uses deprecated Gradle conventions

Sumber: ht

Kotlin Gradle plugin no longer uses deprecated Gradle conventions
Prior to Kotlin 2.0.0, if you used Gradle 8.2 or higher, the Kotlin Gradle plugin incorrectly used Gradle conventions that had been deprecated in Gradle 8.2. This led to Gradle reporting build deprecations. In Kotlin 2.0.0, the Kotlin Gradle plugin has been updated to no longer trigger these deprecation warnings when you use Gradle 8.2 or higher.

[kotlin_144] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Standard library

Sumber: ht

Standard library
This release brings further stability to the Kotlin standard library and makes even more existing functions common for all platforms:
* Stable replacement of the enum class values generic function
* Stable AutoCloseable interface
* Common protected property AbstractMutableList.modCount
* Common protected function AbstractMutableList.removeRange
* Common String.toCharArray(destination)

[kotlin_145] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Stable replacement of the enum class values generic function

Sumber: ht

Stable replacement of the enum class values generic function
In Kotlin 2.0.0, the `enumEntries()` function becomes Stable.
The `enumEntries()` function is a replacement for the generic `enumValues()` function. The new function returns a list of all enum entries for the given enum type `T`. The `entries` property for enum classes was previously introduced and also stabilized to replace the synthetic `values()` function. For more information about the entries property, see What's new in Kotlin 1.8.20.
> The `enumValues()` function is still supported, but we recommend that you use the `enumEntries()` function
> instead because it has less of a performance impact. Every time you call `enumValues()`, a new array is created, whereas
> whenever you call `enumEntries()`, the same list is returned each time, which is far more efficient.
>
{style="tip"}
For example:

Contoh Kode / Perintah CLI:

```
enum class RGB { RED, GREEN, BLUE }

inline fun <reified T : Enum<T>> printAllValues() {
    print(enumEntries<T>().joinToString { it.name })
}

printAllValues<RGB>()
```

```
// RED, GREEN, BLUE
```

[kotlin_146] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Stable AutoCloseable interface

Sumber: ht

Stable AutoCloseable interface
In Kotlin 2.0.0, the common `AutoCloseable`
interface becomes Stable. It allows you to easily close resources
and includes a couple of useful functions:
* The `use()` extension function, which executes a given block function on the selected resource and then closes it down
correctly, whether an exception is thrown or not.
* The `AutoCloseable()` constructor function, which creates instances of the `AutoCloseable` interface.
In the example below, we define the `XMLWriter` interface and assume that there is a resource that implements it.
For example, this resource could be a class that opens a file, writes XML content, and then closes it:

Contoh Kode / Perintah CLI:

```
interface XMLWriter {
    fun document(encoding: String, version: String, content: XMLWriter.() -> Unit)
    fun element(name: String, content: XMLWriter.() -> Unit)
    fun attribute(name: String, value: String)
    fun text(value: String)

    fun flushAndClose()
}

fun writeBooksTo(writer: XMLWriter) {
    val autoCloseable = AutoCloseable { writer.flushAndClose() }
    autoCloseable.use {
        writer.document(encoding = "UTF-8", version = "1.0") {
            element("bookstore") {
                element("book") {
                    attribute("category", "fiction")
                    element("title") { text("Harry Potter and the Prisoner of Azkaban") }
                    element("author") { text("J. K. Rowling") }
                    element("year") { text("1999") }
                    element("price") { text("29.99") }
                }
                element("book") {
                    attribute("category", "programming")
                    element("title") { text("Kotlin in Action") }
                    element("author") { text("Dmitry Jemerov") }
                    element("author") { text("Svetlana Isakova") }
                    element("year") { text("2017") }
                    element("price") { text("25.19") }
                }
            }
        }
    }
}
```

[kotlin_147] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Common protected property AbstractMutableList.modCount

Sumber: ht

Common protected property AbstractMutableList.modCount
In this release, the `modCount`
`protected`property of the `AbstractMutableList` interface becomes common. Previously, the `modCount` property was available on each platform but not for the common target. Now, you can create custom implementations of `AbstractMutableList`
and access the property in common code.
The property keeps track of the number of structural modifications made to the collection. This includes operations that change the collection size or alter the list in a way that may cause iterations in progress to return incorrect results.
You can use the `modCount` property to register and detect concurrent modifications when implementing a custom list.

[kotlin_148] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Common protected function AbstractMutableList.removeRange

Sumber: ht

Common protected function AbstractMutableList.removeRange
In this release, the `removeRange()`
`protected`function of the `AbstractMutableList` interface becomes common. Previously, it was available on each platform
but not for the common target. Now, you can create custom implementations of `AbstractMutableList` and override the function in common code.
The function removes elements from this list following the specified range. By overriding this function, you can take advantage of the custom implementations and improve the performance of the list operation.

[kotlin_149] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Common String.toCharArray(destination) function

Sumber: ht

Common String.toCharArray(destination) function
This release introduces a common `String.toCharArray(destination)`
function. Previously, it was only available on the JVM.
Let's compare it with the existing `String.toCharArray()` function.
It creates a new `CharArray` that contains characters from the specified string. The new common `String.toCharArray(destination)`
function, however, moves `String` characters into an existing destination `CharArray`. This is useful if you already have a buffer that you want to fill:
{kotlin-runnable="true"}

Contoh Kode / Perintah CLI:

```
fun main() {
    val myString = "Kotlin is awesome!"
    val destinationArray = CharArray(myString.length)

    // Convert the string and store it in the destinationArray:
    myString.toCharArray(destinationArray)

    for (char in destinationArray) {
        print("$char ")
        // K o t l i n  i s  a w e s o m e !
```

```
    }
}
```

[kotlin_150] Kotlin - Modern Features - Kotlin 2.0 (K2 Compiler)

What's New in Kotlin 2.0 (K2 Compiler GA, Smart Casts 2.0, Data Objects, Multiplatform): Install Kotlin 2.0.0

Sumber: ht

Install Kotlin 2.0.0

Starting from IntelliJ IDEA 2023.3 and Android Studio Iguana (2023.2.1) Canary 15, the Kotlin plugin is distributed as a bundled plugin included in your IDE. This means that you can't install the plugin from JetBrains Marketplace anymore.

To update to the new Kotlin version, change the Kotlin version
to 2.0.0 in your build scripts.